

Introduction aux logiciels libres

Notes de cours — 10 sessions

Stefane Fermigier

Université Paris-Cité — M1

Table des matières

Session 1 — Introduction aux logiciels libres	1
Objectifs de la séance	1
Pourquoi ce cours ?	1
Partie 1 — Qu'est-ce qu'un logiciel libre ?	1
Partie 2 — Free Software vs Open Source	3
Partie 3 — L'open source aujourd'hui	5
Points de vigilance / pièges fréquents	6
Ce qu'il faut retenir	6
Pour aller plus loin	6
Session 2 — Histoire du logiciel libre (1950-1990)	7
Objectifs de la séance	7
Partie 1 — La préhistoire du logiciel (1950-1970)	7
Partie 2 — Unix : naissance d'un écosystème	8
Partie 3 — La philosophie Unix	8
Partie 4 — La fermeture du logiciel (1970-1983)	10
Partie 5 — Richard Stallman et la naissance du libre	11
Points de vigilance	13
Ce qu'il faut retenir	13
Pour aller plus loin	14
Session 3 — Histoire du logiciel libre (1990-aujourd'hui)	15
Objectifs de la séance	15
Partie 1 — Linux : la pièce manquante (1991)	15
Partie 2 — <i>The Cathedral and the Bazaar</i> (1997)	17
Partie 3 — Le virage Open Source (1998)	18
Partie 4 — L'ère moderne (2000-2020)	19
Partie 5 — L'écosystème du libre	20
État des lieux en 2025	21
Ce qu'il faut retenir	21
Pour aller plus loin	21
Session 4 — Droit et propriété intellectuelle	22
Objectifs de la séance	22
Partie 1 — Pourquoi le droit vous concerne	22
Partie 2 — La propriété intellectuelle	22
Partie 3 — Le droit d'auteur appliqué au logiciel	23
Partie 4 — Qui détient les droits ?	25
Partie 5 — Les brevets logiciels	25
Partie 6 — Les marques	29
Partie 7 — Interopérabilité et reverse engineering	29
Partie 8 — Le copyleft : le « hack juridique »	29
Partie 9 — Qu'est-ce qu'une licence ?	30
Partie 10 — Les grandes familles	31
Ce qu'il faut retenir	31

Pour aller plus loin	31
Session 5 — Maîtriser les licences libres	32
Objectifs de la séance	32
Partie 1 — Les licences permissives	32
Partie 2 — Les licences copyleft	33
Partie 3 — Compatibilité des licences	34
Partie 4 — Choisir une licence	35
Partie 5 — Standards et outils	36
Partie 6 — Contribuer à un projet	36
Partie 7 — Cas spéciaux	37
Partie 8 — Étude de cas : Chardet (2026)	38
Ce qu'il faut retenir	39
Pour aller plus loin	40
Session 6 — Contribuer à un projet open source : fondamentaux	41
Objectifs de la séance	41
Préalable : quelques précisions	41
Partie 1 — Pourquoi contribuer ?	42
Partie 2 — Évaluer un projet	42
Partie 3 — Types de contributions	43
Partie 4 — Structure d'un projet	44
Partie 5 — Le workflow de contribution	44
Erreurs à éviter (résumé)	46
Ce qu'il faut retenir	46
Pour aller plus loin	46
Session 7 — Contribuer efficacement : bonnes pratiques	47
Objectifs de la séance	47
Partie 1 — L'anatomie d'une bonne PR	47
Partie 2 — Messages de commit	48
Partie 3 — Communication	49
Partie 4 — Code review	50
Partie 5 — Pièges à éviter	51
Partie 6 — Démarrer son propre projet	52
Partie 7 — Gestion des versions (<i>release management</i>)	53
Ce qu'il faut retenir	54
Pour aller plus loin	54
Session 8 — Gouvernance des projets open source	55
Objectifs de la séance	55
Deux actualités en guise d'introduction	55
Partie 1 — Le droit au fork	56
Partie 2 — Modèles de gouvernance	56
Partie 3 — Prise de décision	57
Partie 4 — Devenir mainteneur	58
Partie 5 — Les fondations	59
Ce qu'il faut retenir	60
Pour aller plus loin	60
Session 9 — Modèles économiques et open source en entreprise	61
Objectifs de la séance	61
Partie 1 — Le paradoxe de l'open source	61

Partie 2 – Les modèles économiques	62
Partie 3 – Cas d'études et tensions	64
Open source en entreprise	66
Partie 4 – Conformité	66
Partie 5 – L'OSPO (<i>Open Source Program Office</i>)	67
Partie 6 – InnerSource	67
Ce qu'il faut retenir	68
Pour aller plus loin	68
Session 10 – Supply chain et sécurité	69
Objectifs de la séance	69
Partie 1 – La supply chain logicielle	69
Partie 2 – Incidents majeurs	70
Partie 3 – SBOM et inventaire	71
Partie 4 – Gestion des vulnérabilités	74
Partie 5 – Bonnes pratiques	74
Partie 6 – IA et sécurité open source	77
Ce qu'il faut retenir	77
Pour aller plus loin	78

Session 1 — Introduction aux logiciels libres

Objectifs de la séance

- Définir « logiciel libre » et « open source ».
- Énoncer et comprendre les 4 libertés fondamentales.
- Distinguer Free Software et Open Source (philosophies, définitions).
- Identifier les logiciels libres dans son environnement quotidien.

Pourquoi ce cours ?

En 40 ans, le logiciel libre a radicalement transformé la façon dont les logiciels sont **conçus, développés, testés, déployés, vendus et maintenus**. Il structure aujourd'hui des pans entiers de l'industrie : cloud, conteneurs, données, IA. Il est aussi au cœur des débats sur l'**innovation ouverte**, la **souveraineté numérique** (européenne notamment) et l'**économie de l'immatériel**.

Ce cours aborde le logiciel libre non pas sous l'angle technique mais sous ses angles **économiques, juridiques et organisationnels** : comment fonctionnent les projets, comment ils se financent, comment ils sont régis, quelles licences s'appliquent et pourquoi. Le marché associé pèse 6 milliards d'euros en France et plus de 30 milliards en Europe, soient environ 10 % du marché informatique (logiciels et services) professionnel.

Partie 1 — Qu'est-ce qu'un logiciel libre ?

Code source vs binaire

Un logiciel existe souvent^[^1] sous deux formes : le **code source**, lisible par un humain (ou un LLM), écrit dans un langage de programmation, et le **code binaire**, produit par la compilation et exécuté par la machine. Sans le code source, on ne peut ni comprendre précisément ce que fait un logiciel, ni le corriger, ni l'adapter. C'est pour cela que l'accès au code source est une condition de base du logiciel libre, sans être une condition suffisante, comme nous allons le voir.

[^1]: Une exception notable est celle des logiciels interprétés (Python, PHP, JavaScript...).

Propriétaire vs libre

Un **logiciel propriétaire** conserve son code source fermé et impose des restrictions fortes via sa licence : pas de copie, pas de modification, usage conditionné (Windows, Photoshop, Office). Un **logiciel libre** publie son code source et garantit à l'utilisateur le droit de l'utiliser, l'étudier, le copier et le modifier (Linux, Firefox, LibreOffice).

Les 4 libertés fondamentales (FSF)

Un logiciel est **libre** s'il garantit les 4 libertés définies par Richard Stallman et la Free Software Foundation :

N°	Liberté
0	Utiliser le programme, pour n'importe quel usage

N°	Liberté
1	Étudier son fonctionnement (suppose l'accès au code source)
2	Redistribuer des copies
3	Modifier le programme et redistribuer les versions modifiées

Ces 4 libertés sont **indissociables** : si une seule manque, le logiciel n'est pas libre.

Dans une conférence filmée en France en 2012, Richard Stallman déclarait (en Français, *verbatim*):

Je peux expliquer le logiciel libre en trois mots: liberté, égalité, fraternité [...]

Liberté, parce que ce sont des logiciels qui respectent la liberté de leurs utilisateurs.

Égalité, parce que dans la communauté du logiciel libre, tous les utilisateurs sont égaux, personne n'a du pouvoir sur personne.

Et fraternité, parce que nous encourageons la coopération entre les utilisateurs.

Enfin, avec un programme, il n'y a que deux possibilités: ou les utilisateurs ont le contrôle du programme, ou le programme a le contrôle des utilisateurs.

Le premier cas s'appelle le logiciel libre, parce que pour avoir en pratique le contrôle du programme, les utilisateurs ont besoin de plusieurs libertés.

Et ces libertés sont les libertés essentielles qui font la définition, le critère du logiciel libre.

Il y en a quatre. La liberté zéro est la liberté d'exécuter le programme comme tu veux.

La liberté numéro un est la liberté d'étudier le code source du programme et de le changer pour qu'il exécute comme tu veux, pour qu'il fasse ton informatique comme tu veux.

Avec ces deux libertés, l'utilisateur individuel ou l'organisation a le contrôle seul, mais le contrôle d'un seul utilisateur ne suffit pas.

Il faut aussi le contrôle collectif de n'importe quel groupe d'utilisateurs qui veulent coopérer.

Et pour le contrôle collectif, il faut aussi la liberté 2, de diffuser des copies exactes quand tu veux, et la liberté 3, de diffuser des versions modifiées quand tu veux.

Avec ces deux libertés aussi, n'importe quel groupe d'utilisateurs qui veulent coopérer peuvent avoir le contrôle de leur version.

Et donc avec le logiciel libre, nous avons le contrôle individuel et le contrôle collectif en parallèle.

« Free as in freedom, not as in free beer »

Le mot anglais *free* est ambigu : il signifie à la fois « gratuit » et « libre ». Le logiciel libre, c'est la seconde acception.

Conséquences pratiques :

- Un logiciel libre **peut être vendu** : vente de copies, de support, de services. La liberté n'interdit pas le commerce.
- Un logiciel **gratuit n'est pas forcément libre** : Chrome est gratuit mais propriétaire (Chromium, lui, est libre) ; WhatsApp est gratuit mais fermé.

Ce qui n'est *pas* libre — contre-exemples

Catégorie	Exemple	Pourquoi pas libre
Propriétaire classique	Microsoft Office	Code fermé, licence restrictive
Freeware	WhatsApp	Gratuit mais code fermé
Shareware	WinRAR	Code fermé + limitation temporelle
Source available	Elastic (SSPL), BUSL, Confluent	Code visible, licence non libre (restrictions d'usage)

Catégorie	Exemple	Pourquoi pas libre
Open core	GitLab EE, Elastic X-Pack	Noyau libre, extensions propriétaires payantes

Ces catégories reviendront tout au long du cours : elles sont au cœur des stratégies des éditeurs (session 9).

Cas ambigus à connaître

Quelques logiciels omniprésents ont un statut plus subtil que « libre » ou « propriétaire » :

- **VS Code** : le code source (dépôt `microsoft/vscode`) est sous licence MIT, mais les binaires distribués par Microsoft incluent de la télémétrie et des extensions propriétaires, sous une licence **non libre**. Le fork libre sans ces ajouts s'appelle **VSCodium**.
- **Chrome / Chromium** : Chromium est libre (BSD) ; Chrome ajoute des briques propriétaires (codecs, *Widevine*, synchronisation Google).
- **Android / AOSP** : le cœur (Android Open Source Project) est libre (Apache 2.0 + GPL pour le noyau), mais la plupart des appareils embarquent les **Google Mobile Services** propriétaires (Play Store, Maps, Gmail).
- **MongoDB** : libre à l'origine (AGPL), passé en **SSPL** en 2018 — licence considérée non libre par l'OSI et la FSF car elle impose des contraintes aux hébergeurs SaaS.
- **Red Hat Enterprise Linux** : le code est libre (GPL), mais l'accès aux binaires et aux mises à jour est conditionné à un contrat de support payant.

Ces cas limites illustrent pourquoi il faut toujours **lire la licence réelle**, pas se fier à la réputation ou au nom du projet.

Partie 2 — Free Software vs Open Source

Repères historiques (détaillés en session 2)

- **1983** : Richard Stallman lance le **projet GNU** (« GNU's Not Unix ») pour produire un système d'exploitation entièrement libre.
- **1985** : Fondation de la **Free Software Foundation (FSF)**, pour porter juridiquement et politiquement le mouvement.
- **1989** : Première version de la **GNU GPL**, qui formalise juridiquement les 4 libertés et invente le **copyleft** (session 4).
- **1991** : Linus Torvalds publie la première version de **Linux**, le noyau qui manquait à GNU.
- **1998** : Netscape libère le code de son navigateur. Dans la foulée, Eric Raymond, Bruce Perens et d'autres forgent le terme « **open source** » et fondent l'**Open Source Initiative (OSI)** pour le promouvoir auprès des entreprises.

Deux termes, deux philosophies

	Free Software (1983)	Open Source (1998)
Initiateur	Richard Stallman, FSF	Eric Raymond, Bruce Perens, OSI
Angle	Éthique, liberté des utilisateurs	Avantages pratiques, méthodologie
Discours	Mouvement social et politique	Pragmatique, <i>business-friendly</i>
Slogan	« Les utilisateurs méritent la liberté »	« Le code ouvert produit de meilleurs logiciels »

Citation de Stallman : « *Open source is a development methodology ; free software is a social movement.* »
Le terme *open source* a été forgé en 1998 pour rendre l'idée acceptable aux entreprises, qui trouvaient le mot *free* ambigu et Stallman trop clivant.

Les définitions formelles

- **Free Software Definition (FSF)** : les 4 libertés, centrée sur l'utilisateur final.
- **Open Source Definition (OSD, OSI)** : 10 critères, centrée sur la licence et la distribution. Historiquement dérivée des **Debian Free Software Guidelines (DFSG)** rédigées par Bruce Perens en 1997.

En pratique, **les deux définitions sont quasi équivalentes** : dans la quasi-totalité des cas, un logiciel libre est open source, et réciproquement. Pour couvrir les deux, on écrit **FOSS** (*Free and Open Source Software*) ou **FLOSS** (*Free/Libre and Open Source Software*) — le « L » pour *libre* évite l'ambiguïté anglophone.

Les organisations qui incarnent ces définitions

- **FSF** (Free Software Foundation, 1985) — gardienne de la définition du logiciel libre et de la GPL. Très peu de salariés, posture éthique et militante.
 - **FSFE** (Free Software Foundation Europe), **FSFE France**, **APRIL**: pendants européen / français de la FSF (avec parfois des divergences d'opinions sur certains sujets).
- **OSI** (Open Source Initiative, 1998) — gardienne de l'Open Source Definition. Maintient la liste des licences officiellement « open source ».
- **Linux Foundation** (2007, par fusion d'OSDL et du Free Standards Group) — fondation professionnelle hébergeant Linux et des centaines d'autres projets (Kubernetes, Node.js...). Financée par de grandes entreprises, poids lourd du secteur.
- **Apache Software Foundation** (1999), **Eclipse Foundation** — autres fondations généralistes.
- **Python Software Foundation**... — fondations spécialisées par écosystème (gouvernance abordée en session 9).
- **CNLL** (Conseil National du Logiciel Libre) et **APELL** (European Association of Professional Open Source) — représentants professionnels de la filière en France et en Europe.
- Et beaucoup d'autres...

L'Open Source Definition (OSD) — les 10 critères

À retenir comme grille de lecture (on n'apprend pas par cœur) :

1. Libre redistribution (pas de redevance).
2. Code source disponible.
3. Œuvres dérivées autorisées.
4. Intégrité du code source (patches séparés possibles).
5. Pas de discrimination de personnes ou de groupes.
6. Pas de discrimination de domaines d'usage (y compris commercial).
7. La licence s'applique automatiquement, sans accord supplémentaire.
8. Non spécifique à un produit.
9. Non restrictive pour les autres logiciels distribués avec.
10. Neutre technologiquement.

Les critères 5, 6 et 9 sont ceux qu'on invoque le plus souvent pour recaler une licence « presque libre » (ex. clause « pas d'usage militaire » → viole le critère 6).

Quand la distinction importe-t-elle ?

- **Ça compte** pour les discussions philosophiques, le positionnement politique d'une organisation, la communication institutionnelle.
- **Ça ne compte pas** pour le choix technique d'un outil, la licence pratique d'un projet, l'acte de contribuer.

Dans ce cours, on utilise les deux termes de manière interchangeable, sauf mention contraire.

Partie 3 — L'open source aujourd'hui

L'open source est partout

Domaine	Exemples
OS	Linux, Android, FreeBSD
Serveurs web	Apache, Nginx
Bases de données	PostgreSQL, MySQL, MariaDB, SQLite
Langages	Python, Rust, Go
Outils dev	Git, Vim/Neovim, Eclipse
Navigateurs	Firefox, Chromium
Bureautique	LibreOffice, GIMP, Inkscape
Cloud	OpenStack, Docker, Kubernetes
IA/ML	TensorFlow, PyTorch, scikit-learn

Quelques chiffres

- **96 %** des bases de code contiennent du code open source (Synopsys 2023).
- **90 %** des entreprises utilisent de l'open source (Red Hat 2023).
- **100 %** des supercalculateurs du TOP500 et environ **60 %** des smartphones tournent sous Linux.
- **4 %** de parts de marché sur le desktop en France (Linux reste marginal côté poste de travail).
- **GitHub** : plus de 100 millions de développeurs et 400 millions de dépôts — attention, « GitHub » ≠ « open source » (beaucoup de dépôts sont privés, d'autres sans licence libre, et par ailleurs la plateforme elle-même n'est pas libre).

L'open source n'est plus une alternative marginale, c'est le standard de l'industrie.

Qui développe l'open source ?

- **Individus** : bénévoles, étudiants, freelances, passionnés.
- **Organisations** : entreprises (GAFAM, Red Hat, Intel, AMD, NVidia, SUSE, SAP, Siemens...), fondations (Apache, Linux Foundation, Eclipse...), laboratoires et universités, administrations publiques.

Chiffre à retenir : sur le noyau Linux, **plus de 80 %** des contributions viennent de développeurs **payés par des entreprises**. L'image du hacker bénévole isolé correspond de moins en moins à la réalité industrielle — elle reste vraie pour certains projets communautaires (Debian, une grande partie de l'écosystème Python, des bibliothèques critiques comme OpenSSL avant Heartbleed, cf. session 10).

Cette cohabitation entre **contributeurs salariés** et **bénévoles** crée des tensions récurrentes (priorités de développement, roadmap, gouvernance) qu'on reverra en sessions 9 et 10.

Pourquoi contribuer ?

Pour les individus : apprentissage et montée en compétence ; réputation, visibilité, portfolio public ; réseau professionnel ; plaisir et engagement personnel.

Pour les entreprises : réduction des coûts ; innovation partagée et mutualisation ; attraction et recrutement de talents ; influence sur les standards ; éviter le *vendor lock-in*.

Ces motivations seront développées en sessions 7, 9 et 10 (contribution, gouvernance, modèles économiques).

Points de vigilance / pièges fréquents

- **Gratuit ≠ libre** : c'est le premier réflexe à corriger.
- **GitHub ≠ open source** : un dépôt public sans fichier `LICENSE` n'est **pas** libre ; par défaut, le droit d'auteur interdit toute réutilisation. Voir le code ne donne aucun droit en soi.
- « **Source available** » ≠ **libre** : voir le code ne suffit pas ; il faut les 4 libertés (critères OSD). SSPL, BUSL, Elastic License sont *source available*, pas libres.
- **Open core** : la version communautaire est libre, la version entreprise ne l'est pas — ne pas confondre les deux.
- **Android** : le cœur (AOSP) est libre, mais la plupart des appareils embarquent des couches propriétaires (Google Mobile Services).
- « **Open** » **marketing** : « OpenAI », « Open Banking », « API ouverte »... le mot *open* est galvaudé. Toujours vérifier la licence du code, pas le nom du produit.

Ce qu'il faut retenir

1. Un **logiciel libre** garantit 4 libertés : **utiliser, étudier, redistribuer, modifier**.
2. *Free* veut dire **libre**, pas gratuit.
3. **Free Software** et **Open Source** désignent (à d'infimes détails près) les mêmes réalités techniques et juridiques, avec des philosophies différentes — que l'on peut néanmoins considérer comme complémentaires.
4. L'open source est **omniprésent** dans l'industrie logicielle.
5. Tout le monde peut **utiliser** et **contribuer**.

Pour aller plus loin

- Free Software Definition : <https://www.gnu.org/philosophy/free-sw.html>
- Open Source Definition : <https://opensource.org/osd>
- Debian Free Software Guidelines : https://www.debian.org/social_contract#guidelines
- SILL (Socle Interministériel des Logiciels Libres) : <https://code.gouv.fr/sill>
- GNU Manifesto (à lire pour la session 2) : <https://www.gnu.org/gnu/manifesto.html>

Session 2 — Histoire du logiciel libre (1950-1990)

Objectifs de la séance

- Retracer l'évolution de l'industrie du logiciel des années 1950 aux années 1990.
- Expliquer le contexte d'émergence du mouvement du logiciel libre.
- Comprendre la philosophie Unix et son influence durable.
- Identifier les figures fondatrices et leurs contributions.
- Situer les arguments du *GNU Manifesto*.

Partie 1 — La préhistoire du logiciel (1950-1970)

L'ère des mainframes

Dans les années 1950-60, les ordinateurs sont des **machines géantes et coûteuses** (*mainframes*) que seuls gouvernements, grandes entreprises et universités peuvent se payer. Le logiciel est vu comme un **accessoire du matériel** : on achète une machine, le constructeur fournit le système avec.

Les acteurs dominants :

- **IBM**, quasi-monopolistique sur les mainframes.
- **DEC**, qui lance le marché des *miniordinateurs* (PDP-1, PDP-7, PDP-11) à partir des années 1960.
- Les grands centres de recherche : **MIT**, **Berkeley**, **Bell Labs** (AT&T).

Anecdote souvent citée : « *I think there is a world market for maybe five computers.* » — attribuée (à tort) à Thomas Watson, IBM, 1943.

Le logiciel, un bien naturellement partagé

Avant la fin des années 1960, **partager le code est la norme**, et pas le contraire :

- Le logiciel est livré avec le matériel, sans licence restrictive.
- IBM distribue le code source de ses OS.
- Les utilisateurs échangent leurs programmes dans des groupes d'utilisateurs (**SHARE** pour IBM, **DECUS** pour DEC).
- On compare le code à une « recette de cuisine » — on l'améliore, on la passe au voisin.

1969 : le tournant IBM (*unbundling*)

En 1969, IBM annonce son **unbundling** : la séparation commerciale du matériel et du logiciel. Désormais, le logiciel est **facturé séparément**.

Les raisons :

- **Pression antitrust** du gouvernement américain sur IBM.
- **Opportunité commerciale** : la demande existe, IBM la monétise.

Conséquence historique : le logiciel passe du statut de *bien commun* partagé dans un écosystème technique à celui de **marchandise protégée**. C'est le début de l'industrie du logiciel propriétaire telle qu'on la connaît encore.

Partie 2 — Unix : naissance d'un écosystème

Les origines (1969)

Unix naît **aux Bell Labs d'AT&T**, créé par **Ken Thompson** et **Dennis Ritchie**. Contexte :

- 1965 : MIT, Bell Labs et GE lancent **MULTICS**, système ambitieux.
- 1969 : Bell Labs se retire de MULTICS (trop complexe, trop lent).
- Thompson récupère un **PDP-7** inutilisé et écrit un système minimaliste (au départ pour faire tourner son jeu *Space Travel*). Brian Kernighan baptise le tout **UNICS** — jeu de mots avec MULTICS (Uniplexed vs Multiplexed *Information and Computing Service*).
- 1971-1973 : Ritchie conçoit le langage **C** (issu du langage B).
- 1973 : Unix est **réécrit en C**, ce qui le rend **portable** — une innovation majeure à l'époque, où chaque OS était lié à son matériel.

Pourquoi AT&T laisse Unix circuler librement ? AT&T est sous **Consent Decree** (1956) : pour garder son monopole téléphonique, la firme s'est engagée à ne pas commercialiser d'autres produits. Unix est donc distribué aux universités pour quelques centaines de dollars (« prix du média et des frais d'envoi »), avec le code source.

Caractéristiques techniques

Unix apporte plusieurs innovations qui deviendront des standards :

- Noyau simple, modulaire, écrit dans un langage de haut niveau (C).
- **Portable** d'une machine à l'autre.
- **Multi-utilisateur, multi-tâches**.
- Abstraction fondamentale : « *Everything is a file* » — fichiers, périphériques, sockets, processus sont manipulés via la même API.
- Processus, mémoire virtuelle, pipes entre programmes.

Architecture

Trois couches concentriques :

- **Kernel** : gestion des processus, des périphériques, du système de fichiers.
- **Shell** : interface utilisateur en ligne de commande, langage de script.
- **Outils** : l'immense ensemble des utilitaires (`ls`, `grep`, `awk`, `sed`, `make` ...).

Diffusion académique

Comme AT&T ne peut pas le commercialiser, **Unix se répand dans les universités** avec son code source, et devient le **standard académique** dans les années 1970. Cela va avoir deux conséquences majeures : (1) une génération entière d'ingénieurs est formée sur Unix, (2) un fork important naît à **Berkeley** en 1977 : **BSD**.

Partie 3 — La philosophie Unix

Les pipes (1973)

L'invention des **pipes** par **Doug McIlroy** est le geste fondateur de la philosophie Unix. Une pipe (`|`) envoie la sortie d'un programme à l'entrée d'un autre. Exemple :
`cat fichier.log | grep ERROR | sort | uniq -c`.

■ « Was the notion of toolbox there before pipes ? — No. Or did pipes create it ? — **Pipes created it.** »

Les pipes rendent naturel le fait d'écrire de **petits programmes spécialisés**, puis de les **composer** pour résoudre un problème plus large.

Les règles de McIlroy (1978)

1. **Make each program do one thing well.** Pour un nouveau besoin, écrire un nouvel outil plutôt que complexifier un outil existant.
2. **Expect the output of every program to become the input to another.** Sorties structurées, pas de bavardage.
3. **Design and build software to be tried early** — prototyper vite, jeter ce qui ne marche pas.
4. **Use tools** plutôt que de la main-d'œuvre peu qualifiée pour alléger une tâche.

Les règles d'Eric S. Raymond (*The Art of Unix Programming*, 2003)

Énoncés qui formalisent la culture Unix :

- **Modularity** : écrire des pièces simples reliées par des interfaces propres.
- **Clarity** : la clarté prime sur la virtuosité.
- **Composition** : concevoir des programmes qui se connectent à d'autres programmes.
- **Simplicity** : commencer simple, ajouter de la complexité seulement quand c'est indispensable.
- **Transparency** : faciliter l'inspection du comportement.
- **Robustness** : fille naturelle de la transparence et de la simplicité.
- **Least Surprise** : faire ce que l'utilisateur attend.
- **Silence** : si un programme n'a rien d'intéressant à dire, il doit se taire.

« Worse is Better » (Richard Gabriel, 1989)

Dans *The Rise of Worse is Better*, **Richard Gabriel** oppose deux manières de concevoir un logiciel — qu'il nomme d'après les lieux qui les incarnaient à l'époque :

	MIT / Stanford (« The Right Thing »)	New Jersey (Unix, Bell Labs)
Priorités (ordre)	Justesse > cohérence > simplicité d'interface > simplicité d'implémentation	Simplicité d'implémentation > simplicité d'interface > cohérence > justesse
Face à un cas pénible	On complique l'implémentation pour couvrir le cas proprement	On simplifie l'interface, quitte à laisser le cas de côté
Exemple canonique	Lisp Machines, systèmes du MIT AI Lab	Unix, C
Pari implicite	La qualité finira par l'emporter	« <i>Good enough</i> » se diffuse, et s'améliorera ensuite

Pour Gabriel, la thèse contre-intuitive est que **la seconde approche gagne dans la durée**. Un outil « imparfait mais simple » se **porte** plus vite sur de nouvelles machines, se **comprend** plus vite par de nouveaux développeurs, et se **diffuse** plus largement. Cette masse d'utilisateurs et de contributeurs finit par **combler les défauts initiaux** — alors que le concurrent « parfait » reste confidentiel et stagne.

■ « Unix and C are the ultimate computer viruses. » — Richard Gabriel

Ce que le débat recouvre vraiment

Une lecture classique réduit l'opposition à *simplicité contre correction*. **Yossi Kreinin** (*What « Worse is Better » is really about*) montre qu'il y a en fait un **désaccord plus profond, sur le fonctionnement du marché** et des systèmes socio-techniques :

- Les tenants de *The Right Thing* partent du principe que **le marché est défaillant** : il récompense l'inférieur, il faut donc viser la qualité d'emblée, quitte à rester minoritaire.

- Les tenants de *Worse is Better* traitent le marché comme un **mécanisme évolutionnaire** : **survivre** et **se reproduire** (portabilité, compatibilité, adoption) comptent plus que la pureté du design — parce que ce sont précisément les logiciels qui survivent qui sont ensuite améliorés par leurs utilisateurs.

C'est pour cela que l'opposition refait surface régulièrement dans l'histoire de l'informatique : **x86 vs RISC** (la compatibilité descendante et les volumes l'emportent sur l'élégance), **Windows vs Mac** pendant longtemps, **JavaScript** face à des langages plus propres... et surtout, pour ce cours, **Linux vs GNU Hurd** — qu'on va voir en session 3.

Pourquoi c'est essentiel pour comprendre l'open source. Le mouvement du logiciel libre a historiquement pris les deux postures :

- **GNU Hurd** (FSF, 1990) est un projet *Right Thing* : architecture à micro-noyau élégante, ambitieuse, jamais vraiment terminée.
- **Linux** (1991) est un projet *Worse is Better* : noyau monolithique « à l'ancienne », compatible Unix, publié tôt, amélioré par des milliers de contributeurs. **Il a gagné** — pas parce qu'il était techniquement supérieur au départ, mais parce qu'il était suffisamment bon pour être adopté, et qu'il a ensuite accumulé les contributions.

C'est exactement la même logique qu'Eric Raymond formalisera en 1997 sous le nom de « **Cathédrale vs Bazar** » (session 3).

Berkeley et BSD

En 1976-77, Ken Thompson effectue un sabbatique à **UC Berkeley**. **Bill Joy** et **Chuck Haley** démarrent la **Berkeley Software Distribution (BSD)** ; le **CSRG** (*Computer Systems Research Group*) est formellement établi en **1980**, financé par la DARPA.

Contributions majeures de BSD (et pérennes) :

- **Pile TCP/IP** (BSD 4.2, 1983) — base du réseau internet mondial.
- Outils : **vi**, **cs**, **sendmail**.
- **Mémoire virtuelle**, **sockets** réseau.
- Base technique de **SunOS/Solaris**, **NeXTSTEP**, et finalement **macOS** et **iOS**.

BSD adopte une **licence permissive** (« faites ce que vous voulez, créditez-nous ») — un modèle philosophiquement distinct de la GPL (voir session 4).

La culture hacker du MIT

En parallèle, au **MIT AI Lab**, une communauté de programmeurs développe une culture singulière :

- **Partage du code** comme norme absolue.
- **Méritocratie technique** : ce qui compte, c'est la qualité du hack.
- « **L'information veut être libre.** »
- Le système **ITS** (*Incompatible Timesharing System*) n'a même pas de mots de passe : tout est ouvert à tous.

Richard Stallman y entre comme programmeur en 1971. Cette culture constitue son univers de référence ; sa désintégration au début des années 1980 sera le point de bascule.

Partie 4 — La fermeture du logiciel (1970-1983)

L'essor du logiciel commercial

Quelques jalons marquants :

Année	Événement
1975	Fondation de Microsoft (Gates, Allen)
1976	« Open Letter to Hobbyists » de Bill Gates
1976	Fondation d' Apple
1979	VisiCalc , premier <i>killer app</i> sur micro-ordinateur
1980	Amendement du Copyright Act américain : le logiciel est explicitement protégé
1981	IBM PC + MS-DOS
1982	Éclatement d'AT&T : Unix peut désormais être commercialisé

La lettre ouverte de Bill Gates (1976)

Gates s'adresse aux hobbyists (club MITS Altair) qui s'échangent des copies de l'interpréteur BASIC de Microsoft :

« *As the majority of hobbyists must be aware, most of you steal your software. [...] Who can afford to do professional work for nothing ?* »

Ses arguments : le développement logiciel coûte cher, le « piratage » empêche l'innovation, les développeurs méritent d'être payés. Cette lettre marque le **changement de statut symbolique** du logiciel : il devient explicitement une **marchandise**.

L'effondrement du MIT AI Lab

Au début des années 1980, le laboratoire d'IA du MIT se vide :

- Deux **spin-offs** commerciaux absorbent les meilleurs hackers : **Symbolics** et **LMI**.
- Le code qu'ils écrivent devient **propriétaire**.
- La communauté se disloque ; Stallman reste quasiment seul.

Le mythe fondateur : l'imprimante Xerox (vers 1980)

Stallman voulait modifier le pilote d'une **imprimante Xerox 9700** pour qu'elle prévienne quand le papier bourrait. Xerox refuse de lui communiquer le code source : **il n'a plus le droit de réparer l'outil qu'il utilise**.

« *J'ai réalisé que le logiciel propriétaire était une injustice sociale.* »

L'épisode, réel ou largement reconstruit, deviendra le **récit fondateur** du mouvement du logiciel libre : un programmeur empêché d'entraider son propre voisin parce qu'un contrat de licence l'interdit.

Partie 5 — Richard Stallman et la naissance du libre

L'homme

Richard Matthew Stallman (RMS) — né en 1953 à New York, au MIT AI Lab à partir de 1971, brillant développeur (auteur d'**Emacs**), dernier tenant de la culture hacker du laboratoire après son effondrement.

Sa vision se forme à partir de l'expérience Xerox et de la dislocation du MIT :

- Le logiciel propriétaire est une **injustice sociale**, car il empêche l'entraide.
- Les utilisateurs méritent la **liberté**.
- Il faut **reconstruire** un écosystème libre complet, utilisable en production.
- Le sujet est **éthique**, pas simplement technique ou économique.

27 septembre 1983 : l'annonce de GNU

Stallman publie sur Usenet :

« *Starting this Thanksgiving I am going to write a complete Unix-compatible software system called GNU (for Gnu's Not Unix), and give it away free to everyone who can use it.* »

Deux décisions stratégiques clés :

1. **Créer un OS complet** (pas juste un outil isolé).
2. **Être compatible Unix** pour faciliter l'adoption : les utilisateurs existants peuvent migrer sans tout réapprendre.

Le projet GNU

Composants produits, dont plusieurs sont aujourd'hui encore standard :

- **GNU Emacs** (1984) — éditeur extensible.
- **GCC** (1987) — compilateur C (puis C++, Ada, Fortran, etc.).
- **GDB, GNU Make, Bash, Coreutils** (`ls` , `cat` , `grep` , `cp` ...).

Ce qui manque encore à la fin des années 80 : **le noyau**. Le projet **GNU Hurd** est lancé en 1990 mais, pour des raisons de choix techniques ambitieux (architecture à micro-noyau), ne sera jamais véritablement fini. Ce trou sera comblé — fortuitement — par **Linux** (session 3).

Le GNU Manifesto (1985)

Publié dans *Dr. Dobbs's Journal*, le manifeste expose les fondements politiques du projet. Quatre arguments à retenir :

1. Les utilisateurs seront **libres** de partager et modifier leurs outils.
2. Le logiciel **sera meilleur** grâce à la collaboration (intuition qui sera confirmée par *The Cathedral and the Bazaar* en 1997 — session 3).
3. Les développeurs **pourront gagner leur vie** autrement (services, support, formation, adaptation).
4. C'est une question d'**éthique** : le logiciel propriétaire divise les utilisateurs et les prive de leur autonomie.

« *I consider that the Golden Rule requires that if I like a program I must share it with other people who like it.* »

La Free Software Foundation (1985)

Stallman fonde la **FSF** pour donner une structure durable au mouvement :

- Promotion politique et philosophique du logiciel libre.
- Financement du développement de GNU (emploi de développeurs).
- **Défense juridique** des licences libres (notamment GPL).
- Définition de référence : les **4 libertés** (vues en session 1).

1989 : la GPL, un « hack juridique »

La **GNU General Public License** v1 est publiée en 1989 (v2 en 1991, v3 en 2007).

Le mécanisme du copyleft (fondement juridique détaillé en session 4) :

1. Le code est couvert par le **droit d'auteur** (copyright) : Stallman, comme tout auteur, en dispose.
2. La licence **utilise** ce droit pour **garantir** les 4 libertés aux utilisateurs.
3. Toute version modifiée redistribuée doit être publiée **sous GPL**.
4. Les libertés sont donc **virales** et **irrévocables** : elles se propagent avec le code.

■ **Le copyleft retourne le copyright contre lui-même** pour garantir la liberté.

X Window System (1985) — un autre modèle

Le **X Consortium** au MIT publie **X11** sous une licence **très permissive** (ancêtre de la MIT License). Pas de copyleft, pas de viralité, réutilisation quasi libre même dans du propriétaire. C'est l'autre grande tradition du libre, qu'on opposera au copyleft en session 4.

Fin des années 80 : l'écosystème s'étoffe

Année	Événement
1987	GCC 1.0 — compilateur C libre de qualité industrielle
1987	Perl 1.0 (Larry Wall)
1988	X Window System devient libre (MIT License)
1989	GPL v1
1989	Cygnus Solutions — première entreprise commerciale bâtie sur GNU
1990	Début du projet GNU Hurd

Situation en 1990 : tous les composants d'un système libre utilisable existent... **sauf le noyau**. La scène est prête pour le coup de théâtre de 1991 (session 3).

Points de vigilance

- **Attention à l'anachronisme** : dans les années 1960, « partager le code » n'a rien d'un choix militant — c'est simplement la norme technique. Le militantisme apparaît par réaction, après l'*unbundling* et la fermeture progressive.
- **Unix ≠ logiciel libre** : Unix a été distribué largement, mais sous un régime juridique ambigu (licences AT&T). Il n'était pas libre au sens de la FSF. BSD le sera vraiment plus tard (après procès UNIX System Labs vs Berkeley, 1992-94).
- **GNU ≠ Linux** : GNU est l'ensemble des outils utilisateur (compilateur, shell, utilitaires) ; Linux est le noyau. C'est pour cela que Stallman parle de « **GNU/Linux** ».
- « **Free** » reste piégeux en anglais (session 1) : la FSF se bat constamment contre le contresens « gratuit ».

Ce qu'il faut retenir

Année	Événement	Signification
< 1969	Partage du code = norme implicite	Pas d'idéologie : c'est juste la pratique technique de l'époque
1969	<i>Unbundling</i> IBM ; naissance d'Unix (Bell Labs)	Double tournant : le logiciel devient marchandise, un OS portable apparaît
1973	Unix réécrit en C ; invention des pipes	Fondation de la philosophie Unix (modularité, composition)
1976	« Open Letter to Hobbyists » (Gates)	Le logiciel est explicitement affirmé comme marchandise
1977	BSD à Berkeley	Seconde tradition : permissive, universitaire
1980	Copyright Act (US) inclut le logiciel	Cadre juridique de la fermeture

Année	Événement	Signification
1983	Stallman annonce le projet GNU	Réaction militante à la fermeture
1985	FSF créée ; GNU Manifesto	Structure et doctrine du mouvement
1987	GCC 1.0	L'écosystème GNU devient crédible
1989	GPL v1	Invention juridique du copyleft
1990	Écosystème GNU quasi complet, il manque le noyau	La scène est prête pour 1991
1991	(suite en session 3)	

Pour aller plus loin

- GNU Manifesto : <https://www.gnu.org/gnu/manifesto.html>
- Peter Salus, *A Quarter Century of Unix* (1994).
- Eric S. Raymond, *The Art of Unix Programming* (2003) — <http://www.catb.org/~esr/writings/taoup/>
- Eric S. Raymond, *The Cathedral and the Bazaar* — <http://www.catb.org/~esr/writings/cathedral-bazaar/>
- Rob Pike, *Unix History* (conférence, 2018) : https://www.youtube.com/watch?v=_2NI6t2r_Hs
- Brian Kernighan sur Unix (LWN) : <https://lwn.net/Articles/881431/>

Session 3 — Histoire du logiciel libre (1990-aujourd'hui)

Objectifs de la séance

- Expliquer l'émergence de Linux et son modèle de développement.
- Analyser le virage « Open Source » de 1998.
- Identifier les évolutions majeures de l'écosystème de 2000 à aujourd'hui.
- Comprendre la transformation de l'industrie vers l'open source.
- Situer les grandes fondations et événements du libre.

Partie 1 — Linux : la pièce manquante (1991)

Le contexte de 1991

À la fin des années 80, le projet GNU a livré presque tout l'écosystème d'un système libre : compilateur (GCC), éditeur (Emacs), shell (Bash), utilitaires (coreutils), bibliothèque C (glibc). **Il manque le noyau.** Le projet GNU Hurd a été lancé en 1990 mais choisit une architecture à micro-noyau ambitieuse et peine à avancer.

Linus Torvalds

Linus Torvalds, né en 1969 en Finlande, étudiant à l'Université d'Helsinki, est passionné par les systèmes d'exploitation. Il utilise **Minix** (Unix pédagogique d'Andrew Tanenbaum), mais en trouve les limitations frustrantes. Il veut un « vrai » Unix sur son PC 386 et se lance dans un projet personnel, sans grande ambition initiale.

25 août 1991 : l'annonce

Linus poste sur le newsgroup `comp.os.minix` :

« *I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones. [...] It is NOT portable (uses 386 task switching etc).* »

L'ironie est célèbre : « won't be big and professional ». 30 ans plus tard, Linux fait tourner la totalité du TOP500 des supercalculateurs, la majorité des serveurs du cloud et 60 % des smartphones.

Le modèle de développement

Ce qui rend Linux différent, ce sont autant des **décisions techniques** que **sociales** :

Décisions techniques : architecture monolithique (vs micro-noyau) ; compatible POSIX/Unix ; portable au-delà de x86 ; modulaire.

Décisions sociales : publication très précoce du code (« release early, release often ») ; contributions acceptées par email ; méritocratie technique ; Linus comme **BDFL** (*Benevolent Dictator For Life*).

La croissance

Version	Date	Développeurs	Lignes de code
0.01	Sept. 1991	1	10 000
1.0	Mars 1994	100	176 000
2.0	Juin 1996	400	780 000
2.4	Janv. 2001	1 000	3,4 M
5.x	2020+	15 000	28 M

Facteurs de succès : Internet permet la collaboration mondiale ; GNU fournit tout le reste ; compatibilité avec le matériel PC grand public ; la GPL garantit le retour des contributions au pot commun.

GNU/Linux : le système complet

Dès 1992, Linux + les outils GNU forment un OS libre complet et utilisable. Stallman insiste sur « **GNU/Linux** » pour créditer le projet GNU ; Torvalds dit simplement « Linux » et le grand public suit. Le débat continue.

Les distributions

Très vite émerge le concept de **distribution** : un ensemble cohérent comprenant un installateur, un gestionnaire de paquets, des milliers de paquets dérivés de logiciels libres *upstream*, des processus de maintenance et une communauté ou une entreprise qui porte le tout.

Distribution	Année	Fondateur(s)	Particularité
MCC Interim	fév. 1992	Owen Le Blanc (Manchester Computing Centre)	Considérée comme la toute première distribution Linux
SLS (<i>Softlanding Linux System</i>)	mai 1992	Peter MacDonald	Première distribution largement diffusée, avec X11 et TCP/IP
Slackware	juil. 1993	Patrick Volkerding	Dérivée de SLS ; philosophie KISS , configuration manuelle, sans résolution automatique des dépendances ; la plus ancienne encore activement maintenue
Debian	août 1993	Ian Murdock	Communautaire, non commerciale (nom formé sur <i>Debra</i> + <i>Ian</i>) ; <i>Social Contract</i> et DFSG (1997)
Red Hat Linux	oct. 1994	Marc Ewing (+ Bob Young en 1995)	Orientée entreprise ; base du futur RHEL et de Fedora (2003)
SUSE	1994	Hubert Mantel, Roland Dyroff, Burchard Steinbild, Thomas Fehr	Allemagne (société S.u.S.E. GmbH fondée en 1992)

Distribution	Année	Fondateur(s)	Particularité
Mandrake / Mandriva	1998	Gaël Duval	Français ; dérivée de Red Hat, axée grand public (KDE par défaut, installateur simple) ; devenue Mandriva (2005) puis forkée en Mageia (2010) à la liquidation de la société
Gentoo	2002	Daniel Robbins	Compilation locale de chaque paquet (<i>source-based</i>)
Arch Linux	2002	Judd Vinet	<i>Rolling release</i> , minimaliste, configuration entièrement manuelle
Ubuntu	2004	Mark Shuttleworth (Canonical)	Dérivée de Debian ; démocratisation du desktop Linux
Alpine Linux	2005	Natanael Copa	Ultra-légère (musl + BusyBox) ; devenue dominante pour les conteneurs Docker

Aujourd'hui : **Debian, Ubuntu, Fedora, Arch, Alpine** dominant, plus les dérivées cloud/conteneurs.

Partie 2 — *The Cathedral and the Bazaar* (1997)

Eric Raymond

Eric S. Raymond est hacker, anthropologue des communautés techniques, mainteneur de **fetchmail**, auteur du *New Hacker's Dictionary*. Ses convictions politiques (libertarien) le distinguent nettement de Stallman. En 1997, il publie un essai qui va durablement influencer l'industrie.

Deux modèles de développement

La Cathédrale	Le Bazar
Développement fermé	Développement ouvert
<i>Releases</i> espacées	<i>Releases</i> fréquentes
Petit groupe d'experts	Communauté large
Planification rigoureuse	Évolution organique
Ex : GNU Emacs (à l'époque), éditeurs commerciaux	Ex : Linux

Les leçons du Bazar

1. « *Every good work of software starts by scratching a developer's personal itch.* »
2. « *Given enough eyeballs, all bugs are shallow.* » — la **Loi de Linus**.
3. « *Release early. Release often.* »
4. « *Treating your users as co-developers is your least-hassle route to rapid code improvement.* »

5. « *If you have the right attitude, interesting problems will find you.* »

L'impact

L'essai **théorise** ce que Linux faisait intuitivement, **légitime** le modèle de développement ouvert auprès des entreprises, et **fournit un vocabulaire** pour en parler. C'est cet essai qui convaincra Netscape de libérer le code de Navigator en janvier 1998.

Partie 3 — Le virage Open Source (1998)

Janvier 1998 : Netscape libère Navigator

Netscape perd la « guerre des navigateurs » contre Internet Explorer (part de marché passée de 80 % à 20 %). Stratégie désespérée : **libérer le code**. Annonce le 22 janvier, code publié le 31 mars, naissance du projet **Mozilla**, qui deviendra Firefox en 2004.

Février 1998 : le terme « Open Source »

Réunion à Palo Alto. Constat partagé : le terme *free software* est mal compris, la connotation politique effraie les entreprises. **Christine Peterson** suggère « **Open Source** », Raymond et **Bruce Perens** l'adoptent et fondent l'**Open Source Initiative (OSI)**. L'**Open Source Definition** reprend les Debian Free Software Guidelines (1997).

Stallman vs Raymond

Stallman refuse « Open Source »	Raymond défend « Open Source »
Efface la dimension éthique	Plus pragmatique
Réduit le libre à une méthode	Attire les entreprises
« Open source misses the point »	Focus sur les avantages techniques
Continue à dire « Free Software »	Le succès commercial prouve le modèle

Les deux mouvements avancent en parallèle, sur les mêmes logiciels mais avec des philosophies distinctes.

1998-2000 : l'euphorie

Année	Événement
1998	Netscape libéré, OSI créée
1998	Halloween Documents — mémos internes Microsoft fuités révélant leur peur de Linux et leur stratégie « <i>Embrace, extend, extinguish</i> »
1999	IPO spectaculaires de Red Hat et VA Linux (+698 % le premier jour pour VA Linux)
2000	IBM investit 1 milliard \$ dans Linux

Controverses récurrentes du libre

Ces débats reviendront plusieurs fois dans le cours : *Free Software* vs *Open Source*, idéologie vs pragmatisme, noyau monolithique vs micro-noyau (**Torvalds vs Tanenbaum**, 1992), « Linux » vs « GNU/Linux », GPL vs permissif, GNOME vs KDE, le libre face au cloud, inclusivité des communautés, adaptation des définitions aux abus des hyperscalers.

Partie 4 — L'ère moderne (2000-2020)

Consolidation des années 2000

- **Côté serveurs** : Linux domine les serveurs web, **Apache** est leader, **MySQL** et **PostgreSQL** montent en puissance, la stack **LAMP** (Linux + Apache + MySQL + Perl/PHP/Python) devient le standard du web.
- **Côté desktop** : Ubuntu (2004) démocratise Linux, GNOME et KDE s'affrontent, mais Windows reste dominant — « l'année du desktop Linux » devient un thème récurrent.

Outils de développement

Année	Événement
1990	CVS (GPL)
2000	Subversion
2005	Git (Linus Torvalds, après rupture avec BitKeeper)
2005	Mercurial
1999	SourceForge (invente le concept de « forge » logicielle)
2008	GitHub (propriétaire)
2011	GitLab (open core)

2005 : Git révolutionne la collaboration

Linux utilisait **BitKeeper** (propriétaire, gratuit pour les projets libres). Un conflit avec BitMover (éditeur de BitKeeper) entraîne la révocation de la licence. Torvalds développe **Git** en deux semaines. Apports majeurs : développement **distribué**, branches/merges faciles, chaque développeur a une copie complète de l'historique. Git rend possible GitHub (2008).

GitHub : la forge sociale (2008)

GitHub ajoute une couche **sociale** (profil développeur, stars, followers: *social coding*) au-dessus de Git et intègre Pull Requests, Issues, Actions/CI. Résultat : **GitHub devient le lieu de l'open source** — 100 M+ développeurs en 2023. À noter : GitHub **n'est pas libre** lui-même (code fermé, appartient à Microsoft depuis 2018).

Timeline business

Année	Événement
1999	IPO Red Hat, VA Linux
2000	IBM investit 1 Md \$ dans Linux
2006	Red Hat rachète JBoss (350 M \$)
2008	Sun rachète MySQL (1 Md \$)
2009	Oracle rachète Sun
2018	Microsoft rachète GitHub (7,5 Md \$)
2019	IBM rachète Red Hat (34 Md \$)
2021	IPO de GitLab (15 Md \$ de capitalisation)

2010s : Cloud et conteneurs

Année	Événement
2010	OpenStack (Rackspace + NASA)
2013	Docker (origine française) « invente » les conteneurs
2014	Kubernetes (Google)
2015	Création de la CNCF (Cloud Native Computing Foundation)

« Microsoft loves Linux » (2014)

Formule lancée par **Satya Nadella** (nouveau CEO) en octobre 2014 — retournement le plus spectaculaire de l'industrie :

Avant (années 2000) : « *Linux is a cancer* » (Ballmer, 2001) ; offensives brevets contre Android ; *Halloween Documents* ; FUD généralisé contre l'open source.

Après (2014-) : Azure supporte Linux ; **WSL** (Windows Subsystem for Linux) ; **VS Code** publié en open source ; rachat de GitHub ; Microsoft devient membre de la Linux Foundation.

Partie 5 — L'écosystème du libre

Les grandes fondations (sessions 8-9)

Année	Fondation
1985	FSF
1999	Apache Software Foundation
2001	Python Software Foundation
2004	Eclipse Foundation
2007	OW2 ; Linux Foundation (fusion d'OSDL et du Free Standards Group)
2010	Document Foundation (LibreOffice)
2015	CNCF
2021	Rust Foundation

Associations françaises

- **1996** : APRIL — promotion et défense du logiciel libre.
- **1998** : AFUL (Association Francophone des Utilisateurs de Logiciels Libres) ; premiers LUGs (Parinux, GUILDE...) ; LinuxFr.org.
- **2001** : Framasoft — associatif, logiciels libres grand public et éducation.
- **2002** : ADULLACT — collectivités territoriales.
- **2010** : CNLL — Conseil National du Logiciel Libre (filiale professionnelle).

Langages « libres »

- **1987** : Perl — **1991** : Python — **1994** : PHP — **1995** : Ruby.
- **2009** : Go (Google) — **2010** : Rust (Mozilla) — **2015** : Zig.

Question piège : Java n'est pas dans cette liste. OpenJDK est libre depuis 2007 mais Sun l'a longtemps gardé sous contrôle ; la marque et la plateforme restent largement Oracle. Même logique pour Swift ou C#.

Textes fondateurs

- 1984 : Steven Levy, *Hackers: Heroes of the Computer Revolution*.
- 1985 : *GNU Manifesto* (Stallman).
- 1997 : *The Cathedral and the Bazaar* (Raymond).
- 1998 : *Le Hold-Up planétaire* (Roberto Di Cosmo).
- 2020 : *Working in Public* (Nadia Eghbal).

Le Debian Social Contract (1997)

Publié par Bruce Perens. Debian s'engage à : (1) rester 100 % libre, (2) rendre à la communauté, (3) ne pas cacher les problèmes, (4) faire passer les utilisateurs et le logiciel libre avant tout. Ce texte servira ensuite de base à l'**Open Source Definition**.

État des lieux en 2025

Où l'open source domine : serveurs (plus de 90 %) ; cloud (Linux majoritaire chez les *hyperscalers*, y compris Microsoft Azure) ; smartphones (Android) ; embarqué et IoT ; supercalculateurs (100 % du top 500) ; conteneurs et orchestration ; IA/ML (PyTorch, TensorFlow...).

Où le propriétaire résiste : desktop grand public (4 % pour Linux) ; applications métier (ERP, CRM...) ; jeux vidéo AAA ; logiciels de design et de création (Adobe).

Ce qu'il faut retenir

Chronologie

1. **1991** : Linux comble le vide laissé par GNU Hurd.
2. **1997** : *The Cathedral and the Bazaar* théorise le développement ouvert.
3. **1998** : naissance du terme « Open Source » ; libération de Netscape.
4. **2000s** : Linux conquiert les serveurs, Ubuntu démocratise le desktop.
5. **2005-2008** : Git puis GitHub transforment la collaboration.
6. **2010s** : cloud, conteneurs, et même **Microsoft adopte l'open source**.
7. Fin des années 2010 : une nouvelle vague de tensions — cloud, licences *source available*, souveraineté (sessions 9 et 10).

Citations

- « *Software is like sex: it's better when it's free.* » — **Linus Torvalds**
- « *Given enough eyeballs, all bugs are shallow.* » — **Eric Raymond** (Loi de Linus)
- « *Talk is cheap. Show me the code.* » — **Linus Torvalds**
- « *Linux is a cancer.* » — **Steve Ballmer** (2001) — à citer pour mesurer le chemin parcouru.

Pour aller plus loin

- *The Cathedral and the Bazaar* : <http://www.catb.org/~esr/writings/cathedral-bazaar/>
- Halloween Documents : <http://catb.org/esr/halloween/>
- *Working in Public* (Nadia Eghbal, 2020) — sur l'économie des mainteneurs.
- Préparation session 4 : qu'est-ce que le droit d'auteur ? Différence avec un brevet ?

Session 4 — Droit et propriété intellectuelle

Objectifs de la séance

- Expliquer les mécanismes du droit d'auteur appliqués au logiciel.
- Distinguer droit d'auteur, brevets et marques.
- Identifier qui détient les droits selon le statut (salarié, stagiaire...).
- Comprendre le mécanisme du **copyleft**.
- Savoir lire la structure d'une licence logicielle.

Partie 1 — Pourquoi le droit vous concerne

Le droit vous concerne dès que vous **utilisez, contribuez, publiez ou intégrez** du code dans un cadre professionnel. Les risques concrets : une violation de licence est une **contrefaçon** (pénalement sanctionnée), elle engage la responsabilité de l'entreprise, peut rendre un produit non distribuable, et a déjà donné lieu à des **litiges coûteux** (Cisco, Orange, Vizio...).

Les licences libres reposent sur un échange : l'auteur *donne* des droits à l'utilisateur (utiliser, modifier, redistribuer), en échange de quoi celui-ci *respecte* certaines conditions. Formule canonique : « *acceptez les conditions ou refusez le don* ».

■ Ignorer une licence ne vous protège pas ; le droit n'est pas optionnel.

Partie 2 — La propriété intellectuelle

La **propriété intellectuelle (PI)** regroupe les droits exclusifs sur les créations de l'esprit. On la divise traditionnellement en deux branches :

Propriété industrielle : brevets, marques, dessins et modèles, indications géographiques.

Propriété littéraire et artistique : droit d'auteur (*copyright*), droits voisins, droits sur les bases de données.

Le logiciel est protégé principalement par le droit d'auteur, accessoirement par brevets (variable selon les juridictions) et marques (pour le nom et le logo).

Les trois piliers pour le logiciel

Protection	Objet	Durée	Formalité
Droit d'auteur	Expression, code	Vie + 70 ans	Automatique
Brevet	Invention technique	20 ans	Dépôt examiné
Marque	Nom, logo, slogan	10 ans (renouvelable)	Enregistrement

Exemples concrets : le **code source de Linux** est protégé par droit d'auteur ; le **nom « Linux »** est une marque déposée (Linux Foundation) ; un **algorithme de compression** peut, aux USA, être brevetable.

Justification et critiques

La PI repose sur un **échange de type contractuel** : l'État accorde un monopole temporaire à l'inventeur ou à l'auteur, en échange de la divulgation publique et du progrès collectif à terme. Les critiques classiques : création de monopoles temporaires, complexité juridique, coûts d'accès à la connaissance, et parfois frein à l'innovation elle-même (brevets trop larges, *patent trolls*).

Partie 3 — Le droit d'auteur appliqué au logiciel

Principes fondamentaux

Le droit d'auteur protège l'**expression** d'une idée, pas l'idée elle-même. Les œuvres doivent être **originales** (empreinte de la personnalité de l'auteur). Protection **automatique dès la création**, sans dépôt ni formalité. Pas de protection pour : les idées en tant que telles, les concepts, les méthodes, les algorithmes purs, les faits, les données brutes.

■ « *Les idées sont de libre parcours.* » — Henri Desbois

Durée : vie de l'auteur + 70 ans (UE et US pour les personnes physiques). Aux US, les œuvres de commande ou produites par des salariés ont une durée distincte : **95 ans après publication ou 120 ans après création**, au premier des deux termes échu.

Droits moraux vs patrimoniaux

Droits moraux (inaliénables en France) : droit de paternité (être identifié comme auteur), droit au respect de l'œuvre, droit de divulgation, droit de retrait (rare).

Droits patrimoniaux (cessibles) : droit de reproduction, droit de distribution, droit de modification, droit de communication au public.

Les licences libres portent sur les droits patrimoniaux. Les droits moraux subsistent : c'est pourquoi la plupart des licences imposent de **conserver les mentions d'auteur** (même en cas de fork ou de modification).

Textes de référence

- **France** : loi du 3 juillet 1985 (intégrée au Code de la propriété intellectuelle, CPI).
- **Europe** : directive 91/250/CEE, puis 2009/24/CE.
- **États-Unis** : *Computer Software Copyright Act* de 1980 — un amendement au *Copyright Act* de 1976, qui inclut explicitement les logiciels dans le champ du droit d'auteur.

Ce qui est protégé et ce qui ne l'est pas

Protégé : le code source, le code objet (binaire), la documentation, le matériel de conception.

Non protégé (ou débattu) : les APIs et les interfaces (débattu), les fonctionnalités, les langages de programmation, les formats de fichiers.

Cas emblématiques

Trois affaires structurent la jurisprudence moderne du logiciel, présentées ici dans l'ordre chronologique.

Apple vs Microsoft / HP (1988-1994) — le *look and feel* de l'interface Mac

Apple attaque **Microsoft** (Windows 2.0) et **Hewlett-Packard** (NewWave) pour violation de l'« *audio-visual copyright* » de l'interface du Macintosh. Le juge **Vaughn Walker** pose un principe structurant : seuls les **éléments d'expression originaux spécifiques** peuvent être protégés, **pas** le *look and feel* général ni les éléments **fonctionnels** (barre de menus, icônes comme métaphores de fichiers, fenêtres superposables, presse-papier...). Microsoft gagne l'essentiel des griefs en 1994 ; Apple appelle en vain.

Portée : on ne peut pas monopoliser une **convention d'interaction**. C'est exactement la même logique qui sous-tendra *Oracle vs Google* trente ans plus tard sur les APIs.

Affaire SCO vs IBM (2003-2021) — la longue agonie du FUD anti-Linux

La société **SCO Group** (ex-Caldera International, renommée en 2002) prétend avoir acquis de **Novell** les *copyrights* sur Unix. En **mars 2003**, elle attaque **IBM** pour **1 milliard de dollars** : selon SCO, IBM aurait introduit du code Unix protégé dans Linux via AIX et Dynix. Elle étend ensuite ses attaques aux **utilisateurs finaux** (DaimlerChrysler, AutoZone) pour propager la peur dans tout l'écosystème.

Arrière-plan financier : **Microsoft** achète à SCO une « licence Unix » pour environ 10 M\$ en 2003 ; **BayStar Capital** injecte 50 M\$ (prétendument à la suggestion de Microsoft). Beaucoup d'observateurs y voient un financement indirect d'une **guerre juridique contre Linux** à un moment où celui-ci menaçait les parts de marché de Windows Server.

Réaction communautaire exemplaire : le blog **Groklaw**, animé par **Pamela Jones**, documente minutieusement chaque pièce du dossier, décode la jurisprudence pour les non-juristes, et démonte pièce par pièce les prétentions de SCO. C'est l'un des tout premiers cas où une **communauté open source s'organise juridiquement**, par un journalisme d'investigation citoyen distribué.

Effondrement :

- **Août 2007** — le juge **Dale Kimball** tranche : **c'est Novell qui détient les copyrights Unix**, pas SCO. Tout l'édifice juridique s'écroule.
- **Septembre 2007** : SCO dépose le bilan (*Chapter 11*).
- **2010** : un jury confirme que Novell est bien le titulaire.
- **Août 2021** : IBM règle ce qu'il reste du contentieux avec le *trustee* de SCO pour **14,25 M\$** — clôture purement administrative.

Portée. Le cas SCO est le cas d'école du **FUD** (*Fear, Uncertainty, Doubt*) comme stratégie anticoncurrentielle : semer le doute juridique sur un concurrent pour ralentir son adoption, même quand les griefs sont vides. C'est aussi la première grande démonstration qu'une **communauté peut résister juridiquement** à une telle attaque par la transparence documentaire.

Oracle vs Google (2010-2021) — les APIs Java dans Android

Oracle rachète Sun (et donc Java) en janvier 2010, puis attaque Google en août 2010 : Android reprend environ **11 500 lignes de déclarations** (signatures, structure et organisation — la SSO) sur 37 packages Java, sans licence. La procédure est spectaculairement longue :

- 2012, juge **William Alsup** : les APIs ne sont **pas** protégeables par le droit d'auteur.
- 2014, **Federal Circuit** (appel) : les APIs *peuvent* être protégées — inversion.
- 2016, second procès : *fair use* retenu pour Google.
- 2018, **Federal Circuit** : *fair use* rejeté — seconde inversion.
- **Avril 2021, Cour Suprême (6-2, majorité Breyer)** : *fair use*. Quatre arguments : (1) les *declaring code* copiés sont indissociables des idées fonctionnelles qu'ils expriment — donc peu protégeables ; (2) l'usage est **transformatif** (Android = plateforme mobile nouvelle) ; (3) les 11 500 lignes ne représentent que **0,4 %** du code concerné ; (4) pas d'effet de substitution sur le marché de Java. La Cour insiste surtout sur la nécessité de **ne pas stériliser l'écosystème** en permettant la monopolisation des APIs, qui « *limiterait la créativité de futurs programmes* ». Elle **ne tranche pas** la question de fond : elle raisonne « à supposer que les APIs soient protégeables ».

Portée : les APIs *peuvent* l'être, mais leur réimplémentation propre peut être couverte par le *fair use*. Enjeu vital pour l'open source, qui pratique de longue date la réimplémentation (Wine pour Win32, Samba pour SMB/CIFS, glibc pour la libc, ReactOS...).

Note sur le *fair use* : exception du droit américain qui autorise l'usage sans autorisation dans certains cas (usage transformatif, éducatif, parodie, critique...). L'exception pour usage transformatif

n'existe pas de manière strictement équivalente en droit français, où les exceptions sont plus limitées (art. L.122-5 CPI).

Partie 4 — Qui détient les droits ?

Par défaut : l'auteur (personne physique) détient le droit d'auteur.

Le cas du salarié (France, art. L.113-9 CPI)

Règle spécifique au logiciel : les droits patrimoniaux sont **automatiquement cédés à l'employeur**, pour les logiciels créés dans le cadre du contrat de travail.

Cas ambigus à vérifier dans le contrat :

- Projet personnel développé sur le temps libre mais lié au métier ?
- Utilisation de ressources de l'entreprise ?
- Lien direct avec l'activité professionnelle ?

Le cas particulier des stagiaires

Point essentiel : le stagiaire n'est **pas un salarié**. L'article L.113-9 ne s'applique donc **pas**. Le stagiaire **conserve ses droits patrimoniaux** par défaut.

Conséquence : si une entreprise veut récupérer les droits sur le code écrit par un stagiaire, elle doit prévoir une **cession explicite** dans la convention de stage (ou un contrat séparé).

C'est un piège très courant, à connaître si vous faites un stage qui produit du code à valeur économique.

Partie 5 — Les brevets logiciels

Principe

Un **brevet** accorde à son titulaire un **monopole temporaire** (20 ans à compter du dépôt) sur une **invention technique**, en échange de sa **divulgation complète** au public — tout le monde peut lire le brevet, personne ne peut le mettre en œuvre sans licence. C'est le marché de base de la PI : on troque le secret industriel contre un monopole légal et borné dans le temps.

Conditions de brevetabilité :

- **Nouveauté** — l'invention ne doit pas avoir été divulguée au public avant la date de dépôt, où que ce soit dans le monde (*absolute novelty*).
- **Activité inventive** — elle doit ne pas être évidente pour une personne du métier.
- **Application industrielle** — elle doit produire un effet technique reproductible.

Droit conféré : le titulaire peut **exclure** autrui de fabriquer, utiliser, vendre ou importer l'invention brevetée. Ce n'est pas un droit d'usage (on peut tomber sous un brevet sans le savoir en créant soi-même la même chose), c'est un **droit d'interdire**. En contrepartie, des taxes de maintien annuelles.

Pourquoi le logiciel pose un problème spécifique

Les brevets ont été conçus pour les inventions mécaniques ou chimiques — pas pour le code. Trois difficultés propres au logiciel :

1. **Frontière avec les idées abstraites** : un algorithme est un raisonnement mathématique. Or les mathématiques et les idées abstraites ne sont traditionnellement **pas brevetables**. Où tracer la ligne ?
2. **Caractère combinatoire** : un logiciel moderne assemble des milliers de techniques existantes. Si chacune peut être brevetée, le **minage** de brevets (*patent thickets*) devient impossible à auditer. Par construction, n'importe quel programme non trivial risque d'infringer sur des brevets sans que ses auteurs ne le sachent.

3. **Cycle de vie court** : 20 ans est une éternité dans le logiciel. Un brevet qui couvre une technique de 2010 est toujours en vigueur en 2030 alors que tout l'écosystème a bougé.

Ces trois frictions expliquent pourquoi le débat « faut-il breveter le logiciel ? » est structurellement différent de celui du brevet pharmaceutique ou mécanique.

États-Unis : une évolution en dents de scie

La jurisprudence américaine a oscillé trois fois :

- **1972 — *Gottschalk v. Benson*** : la Cour Suprême refuse un brevet sur un algorithme de conversion décimal/binaire. Un algorithme pur n'est pas brevetable.
- **1981 — *Diamond v. Diehr*** : la Cour Suprême accepte un brevet sur un **procédé industriel** (vulcanisation du caoutchouc) qui utilise un algorithme. Ouvre la porte : un algorithme **appliqué à un procédé physique** peut être breveté.
- **1998 — *State Street Bank v. Signature Financial*** : la Cour d'Appel Fédérale valide un brevet sur une **méthode d'affaires** implémentée en logiciel. **Explosion** des brevets logiciels : des dizaines de milliers de dépôts par an, souvent très larges, peu examinés.
- **2010 — *Bilski v. Kappos*** : la Cour Suprême commence à freiner en rejetant un brevet sur une méthode de couverture de risque financier.
- **2014 — *Alice Corp v. CLS Bank International*** : décision majeure. La Cour Suprême établit un **test en deux étapes** : (1) la revendication porte-t-elle sur une idée abstraite ? (2) si oui, contient-elle un « *inventive concept* » qui la transforme en application concrète ? Ce test a **invalidé des milliers de brevets** logiciels et business-method dans la décennie qui a suivi.

Après *Alice*, le paysage US est moins permissif qu'il ne l'a été entre 1998 et 2014, mais les États-Unis restent **de loin** la juridiction la plus favorable aux brevets logiciels dans le monde.

Europe : exclusion théorique, pratique ambiguë

Article 52(2) de la Convention sur le Brevet Européen (CBE, 1973) : les « **programmes d'ordinateur en tant que tels** » sont **exclus** de la brevetabilité, au même titre que les méthodes mathématiques, les méthodes d'affaires et les présentations d'information.

Mais l'**Office européen des brevets (OEB)** a construit une doctrine dite du « **caractère technique** » (*technical effect*) : si un logiciel produit un **effet technique supplémentaire** au-delà de l'interaction normale avec l'ordinateur (ex. : contrôle d'une caméra, compression avec gain d'efficacité mesurable, chiffrement), alors il peut être breveté en tant qu'« invention mise en œuvre par ordinateur » (*computer-implemented invention*, CII). Résultat : **des dizaines de milliers de brevets CII** ont été accordés en Europe, malgré l'exclusion de principe. Le critère « *as such* » est devenu une ligne floue.

La directive 2005 (CII directive) aurait codifié cette pratique de l'OEB. Elle a été **rejetée** par le Parlement européen le **6 juillet 2005** par **648 voix contre 14** — vote historique après une campagne massive de la **FFII** (*Foundation for a Free Information Infrastructure*), portée notamment par des acteurs français (AFUL, April) et européens. Les brevets logiciels n'ont donc pas été explicitement codifiés en droit de l'UE ; la pratique de l'OEB perdure mais reste juridiquement contestée.

Nouveauté : la Juridiction Unifiée du Brevet (JUB / UPC) — opérationnelle depuis le **1^{er} juin 2023**. Elle centralise le contentieux sur les « brevets unitaires européens » dans la plupart des États membres. Effet probable : renforcement de l'exécution (*enforcement*) des brevets en Europe, y compris pour les CII. À surveiller.

Cas emblématiques

Cas	Années	Objet	Issue
Unisys / LZW (GIF)	1994-2004	Algorithme de compression LZW utilisé dans GIF	Unisys réclame des royalties, déclenche la création de PNG comme alternative libre
Amazon 1-Click	1999-2017	Achat en un clic	Validé aux US, rejeté en Europe (manque de caractère technique). Expiré en 2017
MPEG / H.264	1996-	Codecs vidéo	<i>Pool</i> de brevets (MPEG LA) ; royalties par appareil. À l'origine de la pression pour des codecs libres (Theora, VP9, AV1)
MP3 / Fraunhofer	1989-2017	Compression audio	Royalties versées par les encodeurs/lecteurs. Brevets expirés en 2017 — MP3 effectivement libre depuis
NTP vs BlackBerry	2001-2006	Email <i>push</i> mobile	NTP (patent troll) obtient 612,5 M\$ en <i>settlement</i> . Cas emblématique du chantage aux brevets
Apple vs Samsung	2011-2018	<i>Slide-to-unlock</i> , design	Condamnations croisées sur plusieurs continents. Apple récupère finalement 539 M\$
Alice Corp v. CLS Bank	2014	Plateforme d'échange financier	<i>Leading case</i> : invalidation des brevets abstraits mis en œuvre par ordinateur

Ne pas confondre avec *Oracle vs Google* (2021), qui concernait le **droit d'auteur** sur les APIs — pas les brevets.

Patent trolls (NPE / PAE)

Un **patent troll** — terme militant, désignation neutre : **NPE** (*Non-Practicing Entity*) ou **PAE** (*Patent Assertion Entity*) — est une entité qui **ne produit rien**, se contente d'**acheter des portefeuilles de brevets** et d'**assigner en justice** des entreprises opérantes. Modèle économique : extorsion (*settlement*) pour éviter des procès coûteux, même quand le brevet est douteux.

Acteurs emblématiques :

- **Intellectual Ventures** (Nathan Myhrvold, ex-Microsoft, fondé en 2000) — 40 000 brevets en portefeuille à son apogée ; a levé plusieurs milliards auprès de fonds de pension.
- **VirnetX vs Apple** — plusieurs centaines de millions en jugements successifs sur des brevets VPN/FaceTime.

Facteur aggravant US : pendant longtemps, les procès se concentraient dans le **Eastern District of Texas** (tribunaux jugés *plaintiff-friendly*). Le *venue* a été restreint par la Cour Suprême dans **TC Heartland v. Kraft Foods (2017)**, ce qui a fait chuter significativement l'activité des trolls.

Défenses collectives de l'open source

Face au risque, l'écosystème libre s'est organisé :

- **OIN — Open Invention Network** (2005) : *pool* de brevets cross-licencié couvrant le « *Linux system* ». Membres (>4 000) : IBM, Google, Red Hat, SUSE, Sony, Microsoft (!) depuis 2018... Tout membre s'engage à ne pas attaquer les autres sur les brevets du pool.
- **LOT Network** (*License on Transfer*, 2014) : 5 000 membres. Engagement : si l'un vend un brevet à un *troll*, les autres reçoivent automatiquement une licence — neutralise la revente de portefeuilles aux NPE.
- **Unified Patents** : organisation qui conteste les brevets trolls en procédure *inter partes review* (IPR).
- **Pledges** publiques : IBM a *pledge* 500 brevets pour l'open source (2005) ; **Tesla** a ouvert tous ses brevets (2014, déclaration de bonne foi) ; **Red Hat Patent Promise**.
- **Prior art** : bases collectives (ex. *Linux Defenders*) destinées à documenter l'antériorité et à invalider les brevets abusifs.

Épisode marquant : **en 2007, Microsoft prétend que Linux viole 235 de ses brevets**, sans jamais les lister. Classique campagne de **FUD** (cf. cas SCO ci-dessus). La réaction : fondation/renforcement de l'OIN, accord Novell-Microsoft controversé (2006), puis, in fine, adhésion de Microsoft à l'OIN en 2018.

SEP et FRAND : un cas particulier

Les **SEP** (*Standard Essential Patents*) sont les brevets indispensables pour mettre en œuvre un standard (H.264, MP3, 3G/4G/5G, Wi-Fi...). Les organismes de normalisation (**ETSI**, **IEEE**...) exigent que leurs titulaires s'engagent à les licencier à des conditions **FRAND** (*Fair, Reasonable and Non-Discriminatory*). Enjeu pour l'open source : une licence FRAND suppose souvent une **redevance par unité** et un **accord bilatéral**, ce qui est **incompatible** avec la libre redistribution. D'où les tensions récurrentes autour des codecs (H.264, HEVC) et l'effort pour produire des alternatives *royalty-free* (**AV1** par l'*Alliance for Open Media*, Opus pour l'audio).

Débat : arguments pour et contre

Arguments en faveur des brevets logiciels :

- Protection des investissements en R&D — sans monopole temporaire, pourquoi investir ?
- Incitation à la **divulgation** (alternative au secret industriel) ;
- Valorisation des startups (un portefeuille de brevets est un actif cédable) ;
- Outil de négociation (*cross-licensing*) entre grands acteurs.

Arguments contre :

- Les **brevets sont souvent trop larges** et de piètre qualité (examen superficiel, surtout aux US pré-Alice) ;
- **Freinage** de l'innovation incrémentale (rappeler qu'un logiciel moderne combine des milliers de techniques) ;
- Coûts prohibitifs des litiges (millions de \$ par procès) ; les **PME et l'open source** sont les premiers pénalisés ;
- Les **patent trolls** détournent le système ;
- **Argument historique** : l'industrie du logiciel a innové **pendant 40 ans (1960-2000)** essentiellement sans brevets ; la période post-*State Street Bank* n'a pas produit plus d'innovation, au contraire.

Position dominante dans l'écosystème open source : hostilité de principe aux brevets logiciels, défense active par l'OIN, LOT et les *pledges*, et préférence marquée pour les licences à **clause brevets explicite** (Apache 2.0, GPL v3, MPL 2.0) — cf. session 5.

Partie 6 — Les marques

Ce qu'une marque protège

Le **nom**, le **logo**, le **slogan**, l'**identité commerciale**. La marque est une protection **distincte** du droit d'auteur : un logiciel peut être libre mais son nom reste protégé.

Exemples : « Linux » (Linux Foundation), « Firefox » et son logo (Mozilla), « Red Hat » (IBM), « Debian » (SPI).

Implications pratiques

Vous pouvez : forker le code, le modifier, redistribuer votre version.

Vous ne pouvez pas : utiliser le nom original pour votre fork, utiliser le logo, créer une confusion commerciale avec le projet d'origine.

Exemple célèbre : Debian fork Firefox et doit le renommer **Iceweasel** (2006-2016) parce que la politique de marque de Mozilla exige des builds officiels.

Partie 7 — Interopérabilité et reverse engineering

Décompilation et exception d'interopérabilité

La directive européenne 91/250/CEE interdit en principe la décompilation (qui crée une œuvre dérivée), mais prévoit une **exception pour l'interopérabilité** :

- Décompilation autorisée pour obtenir les informations d'interface,
- si elles ne sont pas disponibles autrement,
- uniquement pour les parties nécessaires à l'interopérabilité.

C'est une **spécificité européenne** importante, qui protège la concurrence et empêche les marchés captifs.

Développement *clean room*

Technique pour **créer un logiciel compatible sans contrefaçon** :

- Équipe A (*dirty room*) analyse le logiciel et documente les spécifications.
- Équipe B (*clean room*) implémente, à partir des specs seulement, sans jamais voir le code original.

Puisque les **idées** ne sont pas protégées, et que l'équipe B n'a jamais vu l'expression originale, son code est juridiquement « propre ».

Cas historiques : BIOS IBM-compatibles (Phoenix, AMI) ; **Wine** (API Windows) ; **Samba** (SMB/CIFS). Le cas de Chardet v7 vu en session 5 pose la question pour les réimplémentations par IA.

Partie 8 — Le copyleft : le « hack juridique »

Mécanisme

Le **copyleft** utilise le droit d'auteur pour **garantir** les libertés au lieu de les restreindre :

1. L'auteur détient le *copyright* sur son code.
2. Au lieu d'en faire un outil de restriction, il **accorde** les droits sous conditions.
3. Condition principale : **toute œuvre dérivée redistribuée doit rester sous la même licence**.

4. Les libertés deviennent **virales et irrévocables** ; elles se propagent avec le code.
- Le copyleft retourne le copyright **contre son objectif initial** pour garantir la liberté.

Copyleft vs domaine public

Domaine public	Copyleft
Aucune protection	Utilise la protection du copyright
N'importe qui peut s'approprier et refermer	Conditions de redistribution qui empêchent la fermeture
Pas de garantie de liberté future	Libertés garanties à perpétuité

Sans copyright, pas de copyleft possible. C'est pour cela que la FSF et les juristes du libre défendent des régimes forts de droit d'auteur, par un paradoxe apparent.

Le spectre des licences

Propriétaire Fermé	→	Permissif MIT/BSD	→	Copyleft faible LGPL/MPL	→	Copyleft fort GPL/AGPL
-----------------------	---	----------------------	---	-----------------------------	---	---------------------------

Critère clé : *que se passe-t-il quand vous redistribuez une version modifiée ?* Rien (permissif) → la bibliothèque reste libre (copyleft faible) → **tout** doit rester libre (copyleft fort).

Efficacité juridique

Le copyleft a été testé en justice à de multiples reprises :

- **Busybox vs nombreuses entreprises** (actions de la SFLC puis SFC).
- **GPL-violations.org** (Harald Welte, Allemagne) — nombreuses victoires.
- **Free (Freebox)** (2011, France) — condamné.
- **Orange vs Entr'Ouvert** (2011-2024, France) — Orange condamné à 800 k€ sur la base d'un usage non conforme de la bibliothèque Lasso (GPL v2).
- **SFC vs Vizio** (2021-25) — enjeux sur le droit contractuel vs droit d'auteur.

La plupart des violations se résolvent **hors tribunal** avec restitution du code source.

Partie 9 — Qu'est-ce qu'une licence ?

Définition

Une licence est un **contrat** entre l'auteur et l'utilisateur. Elle définit les droits accordés, les conditions, les limitations, les obligations en cas de redistribution. Ce qu'elle **ne garantit pas** : le prix, la qualité, le support, la maintenance, la pérennité.

Structure type

1. **Préambule** — intentions et philosophie (optionnel).
2. **Définitions** — termes employés (œuvre, code source, distribution...).
3. **Droits accordés**.
4. **Conditions** — ce que vous devez faire en retour.
5. **Limitations** — ce que vous ne pouvez pas faire.
6. **Clause de garantie** — généralement *AS IS*, aucune garantie.
7. **Clause de responsabilité** — limitation de responsabilité de l'auteur.

Pas de licence = pas de droits

■ Un code sans licence **n'est pas libre**, il est **inutilisable**.

Sans licence explicite : le droit d'auteur s'applique par défaut, « tous droits réservés ». Voir le code sur GitHub ne donne aucun droit de le copier, modifier ou redistribuer.

Partie 10 — Les grandes familles

Type	Exemples	Obligation principale
Permissif	MIT, BSD, Apache 2.0	Attribution (conserver copyright)
Copyleft faible	LGPL, MPL	Partager les modifications de la lib
Copyleft fort	GPL, AGPL	Tout le projet dérivé sous même licence

Chaque famille sera étudiée en détail en **session 5**.

Ce qu'il faut retenir

1. Le logiciel est protégé par le **droit d'auteur** — pas besoin de dépôt.
2. Les **brevets logiciels** existent surtout aux US ; exclus « en tant que tels » en Europe.
3. Les **marques** protègent les noms et logos, même pour des logiciels libres.
4. Le **copyleft** utilise le droit d'auteur pour **garantir** les libertés.
5. En entreprise, **les salariés cèdent automatiquement, les stagiaires non** (à négocier).
6. Une licence est un **contrat** : pas de licence = pas de droits.

Tableau récapitulatif des protections

Protection	Objet	Durée	Formalité	Exemples
Droit d'auteur	Expression, code	Vie + 70 ans	Automatique	Code source, documentation
Brevet	Invention technique	20 ans	Dépôt examiné	Algorithme (US)
Marque	Signe distinctif	10 ans renouvel.	Enregistrement	« Linux », logo
Secret	Information	Illimité	Aucune	Code propriétaire

Pour aller plus loin

- Code de la propriété intellectuelle (partie logiciels) : https://www.legifrance.gouv.fr/codes/texte_lc/LEGITEXT000006069414/
- Droit des Logiciels (F. Pellegrini et S. Canevet, 2013): <https://www.puf.com/droit-des-logiciels>
- Livret bleu « Fondamentaux juridique »: https://cnll.fr/media/LivretBleu_Juridique-2eEdition_GT-LogicielLibre_Systematic_Nov2016_web.pdf

Session 5 — Maîtriser les licences libres

Objectifs de la séance

- Lire et interpréter une licence courante (MIT, GPL, Apache).
- Évaluer la compatibilité entre licences.
- Choisir une licence adaptée à un projet.
- Utiliser les standards (SPDX) et outils.
- Comprendre DCO, CLA et les mécanismes de contribution.

Partie 1 — Les licences permissives

Philosophie

■ « *Faites ce que vous voulez avec ce code, tant que vous me créditez.* »

Caractéristiques : très peu de restrictions, aucune obligation de partager les modifications, compatibles avec du code propriétaire, perçues comme *business-friendly*.

Principales : **MIT**, **BSD** (2 ou 3 clauses), **Apache 2.0**, **ISC**.

MIT — la plus simple et la plus populaire

Texte de 170 mots. Droits accordés très larges : utiliser, copier, modifier, fusionner, publier, distribuer, sous-licencier, vendre. Obligations minimales : **conserver le copyright et la licence** dans les copies. Limitations : aucune garantie, aucune responsabilité.

Projets emblématiques : jQuery, **React**, Rails, Node.js, .NET Core.

BSD — 2, 3 ou 4 clauses

Version	Contenu	Compatibilité
BSD 4-cl (années 1980, 4.3BSD)	1. clause « pas de pub avec mon nom » + « mentionner Berkeley dans la pub »	Incompatible GPL
BSD 3-cl (<i>New BSD</i> , 1999)	Berkeley retire la clause de publicité le 22 juillet 1999	Compatible GPL
BSD 2-cl (<i>Simplified</i>)	Encore plus simple, ≈ MIT	Compatible

Projets : FreeBSD, OpenBSD, nginx.

Apache 2.0

Plus complète que MIT/BSD (2 000 mots), avec des protections supplémentaires :

- **Grant explicite de brevets** : quiconque distribue du code Apache 2.0 accorde automatiquement une licence sur les brevets qu'il détient sur ce code.
- **Protection contre les litiges brevets** : toute action en contrefaçon brevetaire contre le projet entraîne perte de la licence pour l'attaquant.

- Définitions claires, fichier **NOTICE** requis si présent, documentation des modifications.

Projets : **Android (AOSP)**, **Kubernetes**, TensorFlow, Swift.

Comparatif

Critère	MIT	BSD 3-cl	Apache 2.0
Longueur	170 mots	220	2 000
Attribution	✓	✓	✓
Protection brevets	✗	✗	✓
Fichier NOTICE	✗	✗	✓
Complexité	Très faible	Faible	Moyenne

Partie 2 — Les licences copyleft

Philosophie

« Vous pouvez utiliser ce code librement, mais vos modifications doivent rester libres. »

Caractéristiques : libertés **virales**, œuvres dérivées soumises à la même licence, empêche la « privatisation » du code, plus restrictif pour l'usage commercial mixte.

Œuvre dérivée vs œuvre collective

Question centrale pour comprendre le copyleft :

Œuvre dérivée	Œuvre collective (agrégation)
Modification, adaptation, transformation	Assemblage d'œuvres indépendantes
L'œuvre originale est <i>incorporée</i>	Chaque partie reste distincte
Le copyleft s'applique à l'ensemble	Le copyleft ne « contamine » pas les autres
Ex : fork, modification du code	Ex : distribution Linux (kernel + apps)

Copyleft fort vs copyleft faible

Copyleft fort (GPL, AGPL)	Copyleft faible (LGPL, MPL)
Tout programme dérivé doit être libre	Seules les modifications de la lib sont concernées
Linker avec du GPL → tout devient GPL	Possible de linker avec du code propriétaire
Perçu comme « contaminant »	Compromis pour bibliothèques

GPL (GNU General Public License)

Créée par Stallman. Trois versions :

- **v1** (1989) — première version.
- **v2** (1991) — la plus utilisée historiquement (**Linux kernel**).
- **v3** (2007) — ajoute grant explicite de brevets, anti-DRM (**anti-Tivoization**), compatibilité Apache 2.0.

Conditions principales : inclure le **code source** avec toute redistribution, conserver la licence et les copyrights, **pas de restrictions supplémentaires**. Si vous distribuez un logiciel basé sur du GPL, **tout doit être redistribué sous GPL**.

Tivoization : TiVo utilisait du Linux (GPL v2) mais verrouillait matériellement ses appareils, empêchant l'utilisateur d'installer ses propres versions modifiées. GPL v3 interdit cette pratique.

AGPL (Affero GPL)

Problème visé : la GPL ne s'applique que lors de la **distribution**. Un service SaaS n'est pas distribué → pas d'obligation de partager le code, même modifié.

AGPL comble la faille : si vous offrez le logiciel **comme service réseau**, vous devez fournir le code source aux utilisateurs, même sans distribution de binaires.

Projets : Mastodon, Nextcloud, Grafana, Bluemind, OnlyOffice (cf. actualité Euro Office, session 8).

LGPL (Lesser GPL)

Copyleft **faible** conçu pour les bibliothèques :

- La bibliothèque elle-même et ses modifications **restent libres**.
- Le programme qui l'utilise peut être **propriétaire**, à condition de linker **dynamiquement** (le plus souvent).

Projets : glibc, Qt (partiellement), GTK.

Le débat du *linking*

Un programme qui utilise une bibliothèque GPL est-il une œuvre dérivée ? Question non tranchée en jurisprudence, position FSF :

Type de linking	Position FSF	Position alternative
Static linking	Œuvre dérivée	Consensus
Dynamic linking	Œuvre dérivée	Débattu (œuvre collective ?)
Import (Python, JS...)	Œuvre dérivée	Très débattu
Appel réseau / API	Non dérivée	Consensus

En pratique : suivre l'interprétation de l'auteur, ou utiliser LGPL/MPL si on veut éviter la question.

MPL (Mozilla Public License)

Copyleft **au niveau du fichier** :

- Les **fichiers modifiés** restent sous MPL.
- Les **nouveaux fichiers** peuvent être propriétaires.
- Compromis clair, compatible Apache 2.0.

Projets : Firefox, Thunderbird.

Partie 3 — Compatibilité des licences

Règle générale

Quand on combine du code sous plusieurs licences, **les obligations se cumulent**. Certaines combinaisons sont impossibles (obligations contradictoires), d'autres forcent la licence finale du résultat.

Permissif → **Copyleft** : **OK**. On peut intégrer du code permissif dans un projet copyleft — ex. MIT → GPL , BSD → GPL , Apache 2.0 → GPL v3 .

Copyleft → Permissif : NON. On ne peut pas réintégrer du GPL dans du MIT ou du propriétaire. Le copyleft « contamine » : dès qu'on intègre du code GPL, le résultat doit rester GPL.

Matrice simplifiée

Source ↓ / Dest →	MIT	Apache 2.0	GPL v2	GPL v3	LGPL	AGPL
MIT	✓	✓	✓	✓	✓	✓
Apache 2.0	✓	✓	✗	✓	✓*	✓
GPL v2	✗	✗	✓	✗*	✗	✗
GPL v3	✗	✗	✗	✓	✗	✓
LGPL	✗	✗	✓	✓	✓	✓
AGPL	✗	✗	✗	✗	✗	✓

* GPL v2 *only* est incompatible avec GPL v3 ; Apache 2.0 est compatible avec LGPL v3 mais pas LGPL v2.1. Une licence GPL « or later » est beaucoup plus souple.

Conseils pratiques

1. **Vérifier la licence** avant d'intégrer du code externe (outil ou lecture directe).
2. **Documenter** les licences de toutes les dépendances.
3. **Éviter** de mélanger GPL v2 *only* avec d'autres licences.
4. **Préférer** les licences avec clause *or later*.
5. En cas de doute : consulter un expert ou un outil (FOSSA, FOSSology...).

Partie 4 — Choisir une licence

Règle d'or : ne créez pas votre propre licence

Pourquoi : risques juridiques (texte non testé), incompatibilité, confusion pour les utilisateurs, travail de rédaction complexe. Recommandation de l'OSI : « *Il y a déjà trop de licences. N'en ajoutez pas.* »

Questions à se poser

1. **Objectif** : que veut-on accomplir ?
2. **Communauté** : qui va contribuer ? Des entreprises ?
3. **Écosystème** : quelles licences utilisent les projets similaires ?
4. **Compatibilité** : avec quoi le code va-t-il être combiné ?
5. **Modèle économique** : monétisation prévue ?

Guide simplifié

Situation	Choix typique
Adoption maximale, y compris en propriétaire	MIT ou Apache 2.0
Garantir que le code reste libre	GPL
Bibliothèque, usage propriétaire accepté, améliorations libres	LGPL
Service web, anti-exploitation SaaS sans contribution	AGPL
Compromis au niveau du fichier	MPL 2.0

Ressource pratique : <https://choosealicense.com> (utile mais simpliste).

Partie 5 — Standards et outils

SPDX (Software Package Data Exchange)

Standard pour identifier et décrire les licences, normalisé **ISO/IEC 5962:2021**.

Identifiants courants : MIT, Apache-2.0, GPL-3.0-only, GPL-3.0-or-later, LGPL-2.1-only, BSD-3-Clause, MPL-2.0.

Usage : dans `pyproject.toml`, `package.json`, `Cargo.toml`, `pom.xml`, et en **header de fichiers source**.

Expressions SPDX

Pour les cas multi-licences :

```
GPL-2.0-only OR MIT                # Au choix
Apache-2.0 AND MIT                 # Les deux s'appliquent
GPL-2.0-or-later WITH Classpath-exception-2.0 # Avec exception
```

Header recommandé

```
# SPDX-License-Identifier: GPL-3.0-or-later
# SPDX-FileCopyrightText: 2026 Your Name <you@example.com>
```

Outils

Outil	Type	Usage
REUSE (FSFE)	Open source	Vérification des métadonnées licence/copyright
licensee	Open source	Détection automatique (utilisé par GitHub)
ScanCode	Open source	Analyse complète
FOSSology	Self-hosted	Audit entreprise
FOSSA	SaaS	Compliance CI/CD
Snyk	SaaS	Sécurité + licences

Partie 6 — Contribuer à un projet

Qui détient le copyright sur les contributions ?

Par défaut : le contributeur. Conséquences pour le projet :

- Des **dizaines ou centaines de détenteurs de droits** sur une base de code.
- **Impossibilité de changer de licence** sans l'accord de tous.
- Difficulté à poursuivre les violations.
- Identification floue des contributeurs.

Trois mécanismes pour clarifier la situation : **DCO, CLA, Copyright Assignment**.

DCO — Developer Certificate of Origin

Déclaration **légère** : en signant son commit (`git commit -s`), le contributeur certifie qu'il a le droit de soumettre ce code sous la licence du projet. Ajoute une ligne `Signed-off-by:` dans le message de commit.

Utilisé par : **Linux kernel**, CNCF, GitLab, Docker. C'est le mécanisme le plus simple et le moins intrusif.

CLA — Contributor License Agreement

Le contributeur **conserve son copyright** mais accorde une **licence large** au projet (souvent large et irrévocable).

Avantages : licence perpétuelle et irrévocable ; permet le *relicensing* ultérieur ; facilite les poursuites en cas de violation ; sensibilise aux questions de PI.

Inconvénients : friction pour les nouveaux contributeurs ; souvent perçu négativement ; paperasse administrative.

Utilisé par : **Apache**, Google, Microsoft.

Copyright Assignment

Le contributeur **transfère la propriété** de son copyright au projet. Le contributeur n'est plus titulaire ; reçoit parfois un *license back*.

Utilisé historiquement par la **FSF** pour les projets GNU. Rare ailleurs (trop intrusif).

Comparatif

Mécanisme	Propriété conservée	Relicensing
Rien	Oui	Impossible sans accord de tous
DCO	Oui	Impossible sans accord de tous
CLA	Oui (licence accordée)	Possible*
Assignment	Non (transfert)	Possible

* Si le CLA le permet explicitement.

GitHub et *inbound* = *outbound*

Les conditions d'utilisation de GitHub précisent : « *Whenever you add Content to a repository containing notice of a license, you license that Content under the same terms.* » En pratique, contribuer via PR sur GitHub = accepter la licence du projet.

Partie 7 — Cas spéciaux

Dual licensing

Un même projet disponible sous **deux licences au choix** (généralement GPL + licence commerciale). Classique : **MySQL**, **Qt**. Nécessite le contrôle du copyright (CLA). Source de revenus (cf. session 9).

Exceptions de licence

Permissions spéciales ajoutées à une licence existante :

- **Classpath Exception** (Java, utilisée par OpenJDK) : autorise à linker du code non-GPL avec les bibliothèques de la plate-forme sans que le tout soit soumis à la GPL.

- **GCC Runtime Exception** : permet de distribuer un exécutable compilé par GCC sans que la **libgcc** (liée à l'exécutable) ne déclenche le copyleft sur le reste du programme.
- **Font Exception** : les documents utilisant une police libre ne sont pas GPL.

Domaine public

Domaine public « naturel » : copyright expiré (70 ans post-mortem). Rare pour le logiciel (trop récent).

Dédication volontaire : **CC0** (Creative Commons Zero), **Unlicense**, **WTFPL**.

Subtilité : en **France** et **Allemagne**, on ne peut pas vraiment renoncer au copyright. D'où l'utilité de CC0 qui accorde une licence très permissive **en fallback**.

Creative Commons — pour le contenu, pas le code

Variante	Signification	Libre ?
CC0	Domaine public	Oui
CC BY	Attribution	Oui
CC BY-SA	Attribution + <i>ShareAlike</i> (copyleft)	Oui
CC BY-NC	Non-commercial	Non
CC BY-ND	No Derivatives	Non

Usage : documentation, assets, datasets. **Pas recommandé pour le code.**

Licences source available (pas open source)

SSPL (MongoDB), BSL (HashiCorp, MariaDB), **Elastic License**, **Commons Clause**. Motivation : protection contre les hyperscalers (« AWS-proofing »), monétisation directe. **Ces licences ne sont pas open source selon l'OSI** — sujet central de la session 9.

Relicensing (changement de licence)

Possible si vous détenez 100 % des droits (auteur unique, salariés d'une même organisation, cession par les contributeurs, CLA qui le permet) OU si la destination est **compatible** (ex : MIT → GPL).

Impossible si des contributeurs multiples n'ont pas donné leur accord explicite, ou si du code tiers sous copyleft est incorporé.

Cas célèbres : MongoDB (AGPL → SSPL), Elastic (Apache 2.0 → SSPL/ELv2), HashiCorp (MPL → BSL), modules Redis. Réaction communautaire systématique : **forks** (OpenSearch, OpenTofu, Valkey...).

Partie 8 — Étude de cas : Chardet (2026)

Les faits

chardet : bibliothèque Python de détection d'encodage, 130 M téléchargements/mois. Mainteneur : Dan Blanchard depuis 12 ans (auteur original : Mark Pilgrim). En 2026, **version 7.0 : réimplémentée par IA (Claude)**, 48× plus rapide, multi-cœurs. Licence changée de **LGPL** → **MIT**. Méthode : l'API et les tests sont fournis à l'IA, sans lire le code source — similarité JPlag < 1,3 %.

La controverse

Position du mainteneur (Dan Blanchard) : il s'agit d'une réimplémentation *clean room* via IA, le code est entièrement nouveau, il n'y a donc pas d'obligation de conserver la LGPL. L'amélioration (performance) justifie le relicensing.

Objection de l'auteur original (Mark Pilgrim) : le mainteneur a été exposé de façon massive au projet existant ; il y avait un contrat social implicite avec les contributeurs ; la LGPL protégeait leurs apports ; le *relicensing* constitue une rupture de confiance.

Question centrale : une réimplémentation par IA est-elle une « œuvre nouvelle » ou une « œuvre dérivée » ?

Trois points de vue

- **Armin Ronacher** (créateur de Flask) — favorable : « *la GPL va contre l'esprit du partage.* » Voit l'IA comme libératrice.
- **Antirez** (créateur de Redis) — favorable : « *GNU a réimplémenté UNIX, Linux a réimplémenté via Minix. L'IA accélère un processus qui a toujours existé.* »
- **Zoë Kooyman** (FSF) — opposée : « *Refuser aux autres les droits que vous avez reçus est profondément antisocial, quelle que soit la méthode.* »

Les enjeux

Arguments pro-réimplémentation : c'est légalement permis ; il y a un précédent historique (GNU a réimplémenté Unix, Linux a réimplémenté Minix) ; l'IA accélère l'innovation ; cela rééquilibre le rapport de force face aux géants de l'industrie.

Arguments anti-réimplémentation : légal ≠ légitime ; la direction du changement (copyleft → permissif) inverse l'esprit du libre ; le contrat social entre contributeurs est brisé ; la protection offerte par le copyleft s'érode.

Ironie notée par plusieurs observateurs : **Vercel a réimplémenté Bash (GPL) en MIT** pour un de ses produits, mais s'est offusqué quand Cloudflare a fait pareil avec Next.js (MIT).

Questions ouvertes

1. Le droit d'auteur protège-t-il les **spécifications** (API, tests, comportement) ?
2. Le code généré par IA est-il **protégeable** ? (pas de jurisprudence claire en 2026)
3. Faut-il un **copyleft de spécification** (TGPL proposé par certains) ?
4. La **vitesse** (5 jours au lieu de 5 ans) change-t-elle la nature du processus ?

Références :

- Discussion GitHub : <https://github.com/chardet/chardet/issues/327>
- *The Ship of Theseus* — Armin Ronacher : <https://lucumr.pocoo.org/2026/3/5/theseus/>
- *AI and OSS licensing* — Antirez : <https://antirez.com/news/162>
- *Legal vs Legitimate* — Hong Minhee : <https://writings.hongminhee.org/2026/03/legal-vs-legitimate/>

Ce qu'il faut retenir

1. **Permissif** (MIT, Apache) → peu de contraintes, adoption maximale.
2. **Copyleft** (GPL) → garantit que le code reste libre.
3. **Copyleft faible** (LGPL, MPL) → compromis pour bibliothèques.
4. **Compatibilité** : permissif → copyleft OK, inverse **NON**.
5. **SPDX** : standard pour identifier les licences.
6. **DCO / CLA** : mécanismes de contribution.
7. **Ne créez jamais votre propre licence.**

Tableau récapitulatif

Licence	Type	Copyleft	Brevets	Usage typique
MIT	Permissive	Non	Non	Libs, frameworks
Apache 2.0	Permissive	Non	Oui	Corporate, cloud
GPL v2	Copyleft fort	Oui	Implicite	Linux kernel
GPL v3	Copyleft fort	Oui	Oui	Outils GNU
AGPL	Copyleft + SaaS	Oui	Oui	Services web
LGPL	Copyleft faible	Partiel	Oui	Bibliothèques
MPL 2.0	Copyleft fichier	Partiel	Oui	Mozilla

Checklist avant de publier un projet

- Fichier `LICENSE` à la racine.
- Identifier SPDX dans `pyproject.toml` / `package.json` / `Cargo.toml`.
- Headers SPDX dans les fichiers source.
- `CONTRIBUTING.md` expliquant DCO/CLA si applicable.
- Liste des dépendances et licences vérifiée.
- Compatibilité des licences validée.

Pour aller plus loin

- Open Source Definition : <https://opensource.org/osd>
- SPDX : <https://spdx.dev>
- REUSE (FSFE) : <https://reuse.software>

Session 6 — Contribuer à un projet open source : fondamentaux

Objectifs de la séance

- Évaluer un projet avant d’y contribuer.
- Identifier les différents types de contributions possibles.
- Naviguer dans la structure d’un projet open source.
- Utiliser le workflow **fork** → **branch** → **PR**.

Préalable : quelques précisions

Avant de plonger dans les détails, trois rappels à garder en tête :

1. **Il n’y a pas que GitHub** — d’autres forges existent.
2. **Il n’y a pas que Git** — d’autres systèmes de versionnement existent.
3. **Tous les projets sont différents** — il faut s’adapter à chaque contexte.

Cette séance utilise GitHub et Git comme exemples car ils dominent l’écosystème actuel, mais les concepts sont transposables.

Les forges logicielles

Une **forge** est une plateforme collaborative pour le développement logiciel.

Forge	Particularité
GitHub	La plus populaire, rachetée par Microsoft (2018), propriétaire
GitLab	Open core, auto-hébergeable, CI/CD intégré
Codeberg	Non-profit (Allemagne), basée sur Forgejo
SourceHut	Minimaliste, workflow par email
Bitbucket	Atlassian (Jira, Confluence)
Gitea / Forgejo	Léger, auto-hébergeable

Fonctionnalités : hébergement des dépôts, gestion des branches et tags, historique, releases, issues, pull/merge requests, wiki, CI/CD, profils sociaux.

Au-delà de Git

Git domine, mais d’autres systèmes existent et méritent d’être connus :

Système	Caractéristique
Mercurial (hg)	Similaire à Git, utilisé par Facebook (historiquement)
Subversion (SVN)	Centralisé, encore utilisé en entreprise
Fossil	Inclut wiki, tickets, forum (SQLite l’utilise)

Système	Caractéristique
Pijul	Basé sur la théorie des patches, gestion des conflits innovante
Jujutsu (jj)	Compatible Git, undo/redo natif, développé par Google

Le workflow (fork → PR) reste conceptuellement similaire ; seules les commandes changent.

Chaque projet est unique

Ce qui varie d'un projet à l'autre : **taille** (1 personne à plusieurs milliers), **gouvernance** (BDFL, comité, fondation — cf. session 8), **outils** (GitHub, mailing-lists, IRC), **processus** (PRs ou patches par email), **standards de code**, **langue de communication**.

Conséquence pratique : toujours lire la documentation, observer avant de contribuer, s'adapter. Ne pas imposer ses habitudes. Un projet Linux kernel ≠ un projet npm de 100 lignes.

Partie 1 — Pourquoi contribuer ?

Motivations personnelles

Pour vous : apprendre de nouveaux langages et outils ; travailler sur du code de qualité ; construire un portfolio public ; développer son réseau ; résoudre un problème qu'on rencontre soi-même.

Pour votre carrière : GitHub comme « CV technique » ; visibilité dans la communauté ; recrutement direct par les mainteneurs ou leurs employeurs ; preuve tangible de ses compétences.

Recherche académique

Une étude de Gerosa et al. (2021) identifie **10 catégories** de motivations :

Catégorie	Description
Intrinsèque	Plaisir, challenge intellectuel
Altruisme	Aider la communauté
Réputation	Reconnaissance par les pairs
Carrière	Opportunités professionnelles
Apprentissage	Acquérir des compétences
Utilité	Résoudre son propre problème
Sociale	Appartenance à une communauté
Idéologique	Valeurs du libre
Rémunération	Payé pour contribuer
Fun	Gamification, badges

Source : *The Shifting Sands of Motivation*, Gerosa et al., 2021 — <https://arxiv.org/abs/2101.10291>.

Partie 2 — Évaluer un projet

Tous les projets ne sont pas « contribuable ». Avant de commencer, vérifier :

1. **Activité** — le projet est-il vivant ?
2. **Communauté** — y a-t-il d'autres contributeurs ?
3. **Accueil** — le projet accueille-t-il les nouveaux ?
4. **Documentation** — peut-on comprendre comment contribuer ?

5. Réactivité — les PRs sont-elles traitées ?

Indicateurs de santé

Bons signes : commits récents (< 1 mois) ; issues traitées régulièrement ; PRs mergées en temps raisonnable ; CONTRIBUTING.md à jour ; labels « good first issue » ; communication active et respectueuse.

Mauvais signes : dernier commit remontant à plus d'un an ; centaines d'issues ouvertes sans réponse ; PRs en attente depuis des mois ; mainteneur unique silencieux ; pas de documentation ; conflits personnels non résolus visibles dans les fils de discussion.

Métriques à vérifier

Métrique	Où la trouver	Seuil indicatif
Dernier commit	Page principale	< 3 mois
Contributors actifs	Insights > Contributors	> 2-3 actifs récemment
Issues ouvertes	Tab Issues	Ratio fermées/ouvertes > 1
PRs ouvertes	Tab Pull requests	< 50 en attente
Temps de merge	PRs fermées	< 2 semaines médian
Stars	Page principale	Pas de seuil, contexte

Le README comme porte d'entrée

Un bon README contient : description du projet, installation / *quick start*, usage basique, comment contribuer (ou lien vers CONTRIBUTING.md), licence. Un README pauvre signale souvent un projet **pas prêt pour les contributions externes**.

Partie 3 — Types de contributions

Ce n'est pas que du code

Contributions techniques : correction de bugs, nouvelles fonctionnalités, tests, refactoring, optimisations de performance.

Contributions non techniques : documentation, traduction, design / UX, triage d'issues, réponse aux questions (forum, chat), organisation d'événements.

■ On estime que **80 % des contributions open source ne sont pas du code**.

Par où commencer

Pour débiter, privilégier les contributions à faible risque et forte valeur d'apprentissage :

1. **Typos et documentation** — risque faible, impact visible.
2. **Good first issues** — bugs simples, bien documentés par les mainteneurs.
3. **Triage** — reproduire des bugs signalés, ajouter des informations.
4. **Tests** — augmenter la couverture.
5. **Traduction** — si on parle une langue pour laquelle il manque une traduction.

Trouver des *good first issues*

Sur GitHub, chercher les labels `good first issue`, `help wanted`, `beginner friendly`.

Agrégateurs utiles :

- <https://goodfirstissue.dev>
- <https://up-for-grabs.net>

- <https://firstcontributions.github.io>
- <https://github.com/MunGell/awesome-for-beginners>

Partie 4 — Structure d'un projet

Fichiers standards

Fichier	Rôle
README.md	Présentation, installation, usage
LICENSE	Licence du projet
CONTRIBUTING.md	Guide de contribution
CODE_OF_CONDUCT.md	Règles de comportement
CHANGELOG.md	Historique des versions
.github/ISSUE_TEMPLATE/	Gabarits pour les issues
.github/PULL_REQUEST_TEMPLATE.md	Gabarit pour les PRs

CONTRIBUTING.md

Ce fichier explique **comment** contribuer. Contenu typique : comment signaler un bug, comment proposer une fonctionnalité, comment soumettre une PR, standards de code, processus de revue, DCO/CLA si applicable.

■ **Toujours le lire avant de contribuer.**

CODE_OF_CONDUCT.md

Définit les comportements acceptables dans la communauté. Standard le plus répandu : **Contributor Covenant**. Autres : Citizen Code of Conduct, Ubuntu Code of Conduct. Objectifs : créer un environnement inclusif, prévenir les comportements toxiques, définir les recours en cas de problème.

Partie 5 — Le workflow de contribution

Avant de coder : « réserver » l'issue

■ **Règle importante** : ne jamais commencer à coder sans prévenir.

Pourquoi : éviter le travail en double, montrer son intérêt, obtenir des conseils avant de commencer.

Comment :

1. Vérifier que personne n'y travaille déjà (commentaires, *assignee*).
2. Commenter : « *I'd like to work on this, is it still available ?* »
3. Attendre confirmation (ou démarrer si pas de réponse après quelques jours).

Vue d'ensemble

- | | |
|-----------|--|
| 1. Fork | → Copier le dépôt sur votre compte |
| 2. Clone | → Télécharger localement |
| 3. Branch | → Créer une branche pour votre travail |
| 4. Code | → Modifications |
| 5. Commit | → Enregistrer vos changements |
| 6. Push | → Envoyer sur votre fork |
| 7. PR | → Proposer vos changements au projet |

- 8. Review → Répondre aux commentaires
- 9. Merge → Vos changements sont intégrés

Étapes 1-2 : Fork et Clone

Sur GitHub, **Fork** via le bouton en haut à droite (crée une copie sur votre compte). En local :

```
git clone git@github.com:VOTRE-USERNAME/projet.git
cd projet
git remote add upstream git@github.com:ORIGINAL/projet.git
```

Convention : `origin` pointe sur votre fork, `upstream` sur le dépôt original.

Étape 3 : Créer une branche

Toujours travailler sur une branche dédiée, jamais sur `main`.

```
git checkout main
git pull upstream main

git checkout -b fix/typo-in-readme
```

Conventions de nommage typiques : `fix/...`, `feature/...`, `docs/...`, `refactor/...`.

Étapes 4-5 : Code et Commit

Règle d'or : **commits atomiques** — un commit = un changement logique.

```
git status
git diff
git add fichier_modifie.py
git commit -m "Fix typo in installation instructions"
```

Message de commit clair : première ligne < 50 caractères (résumé), ligne vide, corps explicatif si nécessaire. Détails approfondis en session 7.

Étapes 6-7 : Push et Pull Request

```
git push origin fix/typo-in-readme
```

Sur GitHub : aller sur son fork, cliquer *Compare & pull request*, remplir le gabarit, expliquer le changement, soumettre.

Étapes 8-9 : Review et Merge

Ce qui peut arriver :

Feedback positif : « LGTM » (*Looks Good To Me*), merge rapide, remerciements.

Demandes de changements : suggestions de style, demande de tests supplémentaires, questions sur l'approche, refus motivé.

Patience : les mainteneurs sont souvent bénévoles, le temps de réponse peut être long.

Garder son fork à jour

Votre fork diverge du projet original au fil du temps. Synchroniser régulièrement :

```
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

Synchroniser **avant** de créer une nouvelle branche. Le rebase et la gestion des conflits sont traités en session 7.

Erreurs à éviter (résumé)

- Commencer à coder sans discuter (sur les grosses features, toujours discuter d'abord).
- PR massive, inreviewable, qui mélange plusieurs sujets.
- Ignorer le `CONTRIBUTING.md`.
- Disparaître après avoir soumis la PR.
- Prendre les critiques personnellement.

Ces thèmes sont repris et approfondis en **session 7**.

Ce qu'il faut retenir

1. **Évaluer** le projet avant de contribuer (activité, accueil, documentation).
2. **Commencer petit** : typos, documentation, *good first issues*.
3. **Lire** `CONTRIBUTING.md` et `CODE_OF_CONDUCT.md` avant toute chose.
4. **Workflow** : Fork → Branch → Code → PR.
5. **Patience et respect** envers les mainteneurs bénévoles.

Pour aller plus loin

- *First Contributions* : <https://github.com/firstcontributions/first-contributions>
- *Good First Issue* : <https://goodfirstissue.dev>
- *How to Contribute to Open Source* (GitHub) : <https://opensource.guide/how-to-contribute/>
- Karl Fogel, *Producing Open Source Software* : <https://producingoss.com> — référence complète, chapitre « Getting Started ».
- Guide Mozilla : <https://mozilla.github.io/open-leadership-training-series/>
- Open Source Masterclass (MOOC) : <https://opensourcemasterclass.org/>

Session 7 — Contribuer efficacement : bonnes pratiques

Objectifs de la séance

- Rédiger une pull request de qualité.
- Écrire des messages de commit clairs et normalisés.
- Communiquer efficacement avec les mainteneurs.
- Participer à la *code review* (recevoir et donner).
- Lancer son propre projet open source.
- Gérer les versions et les *releases*.

Partie 1 — L'anatomie d'une bonne PR

Qu'est-ce qu'une bonne PR ?

Caractéristiques d'une PR qui sera mergée rapidement :

Caractéristiques à viser : un seul changement logique ; taille raisonnable (< 500 lignes) ; bien documentée ; testée ; conforme aux standards du projet.

Objectifs sous-jacents : faciliter la review, minimiser les risques de régression, accélérer le merge, respecter le temps des reviewers.

Structure d'une PR

Titre : court, descriptif, au présent.

-  « *Fix memory leak in image parser* »
-  « *Fixed stuff* »

Description — modèle simple :

```
## What does this PR do?  
[Description claire du changement]  
  
## Why is this change needed?  
[Contexte, référence à l'issue]  
  
## How to test?  
[Instructions pour tester]  
  
## Screenshots (if applicable)  
[Captures d'écran]
```

Bonnes pratiques pour les PRs

1. Une PR = un changement — pas de PR fourre-tout.
2. Référencer l'issue — Fixes #123 ou Closes #456 pour la fermer automatiquement.
3. Petites PRs > grosses PRs — plus faciles à reviewer.

4. **Expliquer le pourquoi** — le code montre le *quoi*, la description montre le *pourquoi*.
5. **Auto-review avant de soumettre** — relire son propre diff.
6. **Tests inclus** — prouver que ça marche.

Ce qui ralentit une PR

Problèmes courants	Solutions
PR trop grosse	Découper en plusieurs PRs
Pas de description	Utiliser le gabarit du projet
Changements non liés mélangés	Une PR par sujet
Tests manquants	Ajouter les tests
Conflits avec <code>main</code>	Rebaser régulièrement

Partie 2 — Messages de commit

Pourquoi c'est important

- **Historique lisible** pour comprendre l'évolution du projet.
- **Debugging** : `git bisect`, `git blame`.
- **Changelog automatique** : génération des *release notes* à partir des messages.
- **Communication** avec l'équipe — et avec votre futur vous.

Structure canonique

```
<type>(<scope>): <subject>
<ligne vide>
<body>
<ligne vide>
<footer>
```

Exemple :

```
fix(parser): handle empty input gracefully

Previously, passing an empty string would cause a crash.
Now it returns an empty result.

Fixes #42
```

Conventional Commits

Standard de plus en plus adopté — simplifie les *changelogs* automatiques et le *semver* (cf. partie 7).

Type	Usage
<code>feat</code>	Nouvelle fonctionnalité
<code>fix</code>	Correction de bug
<code>docs</code>	Documentation uniquement
<code>style</code>	Formatage, pas de changement de code
<code>refactor</code>	Refactoring sans changement de comportement

Type	Usage
test	Ajout / modification de tests
chore	Maintenance, dépendances, CI

Spécification : <https://www.conventionalcommits.org>.

Règles d'or

1. **Première ligne < 50 caractères.**
2. **Impératif** : « *Add* », pas « *Added* » ou « *Adds* ».
3. **Pas de point à la fin** du sujet.
4. **Corps à 72 caractères** par ligne.
5. **Expliquer le pourquoi** dans le corps.
6. **Référencer les issues** dans le *footer*.

Exemples

Mauvais	Bon
fix bug	fix(auth): prevent session timeout during file upload
WIP	feat(api): add rate limiting to public endpoints
changes	docs: update installation instructions for Windows

Partie 3 – Communication

La communication asynchrone

L'open source fonctionne en **asynchrone** : fuseaux horaires, bénévoles, temps de réponse variables. Implications : être clair et complet dès le premier message, ne pas attendre de réponse immédiate, éviter les *ping* trop fréquents.

Règles

À faire : être poli et respectueux ; fournir le contexte nécessaire dès le premier message ; poser des questions claires ; accepter le feedback ; remercier les reviewers.

À éviter : ton agressif ou impatient ; messages du type « ça marche pas » sans détails ; relancer trop vite ; prendre les critiques personnellement ; argumenter indéfiniment.

Anti-patterns

- **Don't ask to ask** — « *Quelqu'un peut m'aider ?* » → poser directement la question avec le contexte.
- **XY Problem** — demander comment faire X alors qu'on veut Y → expliquer l'objectif réel.
- **Help vampire** — demander de l'aide sans effort préalable → montrer ce qu'on a déjà cherché.

Comment poser une bonne question

```
## Ce que j'essaie de faire
[objectif]

## Ce que j'ai essayé
[tentatives]

## Ce qui se passe
```

```
[comportement actuel, erreurs, logs]
```

```
## Ce que j'attendais  
[comportement souhaité]
```

```
## Environnement  
[versions, OS, configuration]
```

Partie 4 — Code review

Recevoir une review

La *code review* **n'est pas une attaque personnelle**. Mindset : le reviewer essaie d'améliorer le projet, vous apprenez à chaque review, les mainteneurs ont une vision globale, un refus n'est pas un échec.

Types de commentaires

Type	Signification	Réaction
Suggestion	« <i>Consider doing X</i> »	Évaluer, appliquer si pertinent
Question	« <i>Why did you...?</i> »	Expliquer votre raisonnement
Nitpick (nit:)	Détail mineur	Corriger sans discuter
Bloquant	« <i>This must be changed</i> »	Obligatoire avant merge
Praise	« <i>Nice approach!</i> »	Dire merci

Répondre aux commentaires

Pour chaque commentaire : lire attentivement, comprendre le point soulevé, **répondre OU corriger OU discuter**.

Exemples de réponses :

- « *Good point, fixed in abc123* »
- « *I chose this approach because... What do you think?* »
- « *I see your point, but X because Y. Happy to change if you prefer.* »

La review comme contribution

La *code review* **n'est pas réservée aux mainteneurs**. Vous pouvez contribuer en reviewant les PRs des autres, même sans droits de commit.

Dans le projet Subversion, Greg Stein s'est mis à reviewer **chaque commit** qui entrait dans le dépôt, sans jamais demander aux autres de faire pareil. Rapidement, d'autres contributeurs ont suivi. — Karl Fogel, *Producing Open Source Software*.

Avant de reviewer : vérifier que le projet **accueille** les reviews externes, comprendre les attentes.

Donner une review

Soyez constructif : expliquer le pourquoi de chaque commentaire, proposer des alternatives concrètes, reconnaître le bon travail, distinguer clairement bloquant et simple suggestion.

Évitez : commentaires vagues, ton condescendant, *bikeshedding* (détails infinis sur des sujets mineurs), bloquer une PR sans explication.

Partie 5 — Pièges à éviter

Erreurs fréquentes

1. **Commencer à coder sans discuter** — pour les grosses features, discuter d’abord dans l’issue.
2. **PR massive** — impossible à reviewer, souvent abandonnée.
3. **Ignorer les guidelines** — lire `CONTRIBUTING.md`.
4. **Push force sans prévenir** — peut casser la review en cours.
5. **Disparaître après la PR** — répondre aux commentaires.

Avant de commencer à coder

1. **Vérifier que l’issue n’est pas prise** — quelqu’un y travaille-t-il déjà ? Y a-t-il une PR en cours ou abandonnée ?
2. **Signaler votre intention** — commenter : « *I’d like to work on this* ».
3. **Synchroniser le fork** — `git fetch upstream && git rebase upstream/main` avant de créer la branche.
4. **Comprendre le contexte** — lire les discussions, vérifier les PRs liées.

Garder son fork à jour

Pourquoi c’est important : éviter les conflits massifs, baser le code sur la version courante, s’assurer que les tests passent.

Moment	Obligatoire ?
Avant de créer une branche	✓
Pendant le travail	Recommandé
Avant de soumettre la PR	✓
Quand le reviewer le demande	✓

Le `Signed-off-by` et le DCO

Certains projets (Linux, CNCF) exigent un DCO. Un `git commit -s -m "..."` ajoute automatiquement :

```
Signed-off-by: Your Name <your@email.com>
```

Signification : « *je certifie avoir le droit de soumettre ce code.* » Détails en session 5.

Gérer un rebase

Votre PR a des conflits avec `main` ?

```
git fetch upstream
git rebase upstream/main

# Résoudre les conflits
git add .
git rebase --continue

git push --force-with-lease origin ma-branche
```

Préférer `--force-with-lease` à `--force` : refuse de pousser si quelqu’un d’autre a poussé entre temps.

Partie 6 — Démarrer son propre projet

Quand ouvrir son code ?

Contextes courants : application personnelle ou académique ; bibliothèque réutilisable ; outil non-cœur-de-métier d'une entreprise ; logiciel livré à un client sans besoin d'exclusivité ; produit principal d'un éditeur (modèle Red Hat).

Motivations possibles : partager (générosité) ; recevoir des contributions extérieures ; rendre le code auditable ; attirer des développeurs et recruter ; créer un standard commun avec ses pairs.

Ouvrir tôt, pas « quand ce sera prêt »

■ Le projet ne sera **jamais** suffisamment prêt.

Pourquoi ne pas attendre : plus on attend, plus le nettoyage est lourd ; on peut recevoir des contributions plus tôt qu'imaginé ; ouvrir le code **n'oblige pas** à écrire une documentation exhaustive, à être disponible en permanence, ni à accepter toutes les contributions.

Avant de recevoir des contributions : choisir une licence (session 5).

Checklist de lancement

Étape	Détail
Choisir une licence	Commencer permissif (MIT) en cas de doute — on peut toujours ajouter de la restriction plus tard
Créer le dépôt	Sur une forge (GitHub, GitLab, Codeberg...)
<code>README.md</code>	Mission en une phrase, résumé en un paragraphe, installation, usage
<code>LICENSE</code>	Fichier avec le texte complet
<code>CONTRIBUTING.md</code>	Comment contribuer, standards, processus
<code>.gitignore</code>	Adapté au langage / framework
CI/CD	Au minimum : lancer les tests automatiquement

Choix technologiques

Règle clé : réduire les besoins d'apprentissage.

Moins il faut apprendre de nouvelles choses pour contribuer → plus il y a de contributeurs potentiels, et plus vite ils sont productifs. En pratique :

- Langage, framework, outils de build : **standards** de l'écosystème.
- Éviter les solutions « maison » sans justification.
- Utiliser les outils que la communauté cible connaît déjà.

Documentation progressive

Ne pas essayer de tout documenter d'un coup. Procéder par niveaux :

1. **Mission en une phrase** — « *X is a Y that does Z* ».
2. **Résumé en un paragraphe** — problème résolu, public cible.
3. **Quick start** — installation et premier usage en 5 minutes.
4. **Exemples** — cas d'usage concrets, démos, captures.
5. **Référence** — API, configuration, options.
6. **Guide du développeur** — architecture, comment contribuer au code.

■ La documentation **utilisateur est essentielle** ; la documentation **développeur est idéale**.

Annoncer le projet

Progressivement :

1. En parler à des connaissances intéressées.
2. Répondre à des besoins exprimés sur des forums en mentionnant le projet.
3. Annonce officielle lors de la première *release*.

Canaux : mailing-lists spécialisées, Reddit, Hacker News, Lobsters, forums du langage/framework, Mastodon, conférences, meetups.

Partie 7 – Gestion des versions (*release management*)

Pourquoi produire des versions ?

Le code est déjà sur un dépôt public. Alors pourquoi des versions numérotées ?

- Désigner une version **recommandée**.
- Communiquer les **changements**.
- Distribuer via les **gestionnaires de paquets** (npm, pip, apt...).
- Produire des **installeurs** / exécutables.
- Permettre aux utilisateurs de choisir **quand mettre à jour**.

Versionnement sémantique (*semver*)

Format : **MAJEUR.MINEUR.CORRECTIF** (ex : 3.12.7).

Incrément	Quand ?	Exemple
MAJEUR	<i>Breaking change</i>	2.0.0 → 3.0.0
MINEUR	Nouvelle fonctionnalité, compatible	3.1.0 → 3.2.0
CORRECTIF	Bug fix, compatible	3.2.1 → 3.2.2

Règles importantes :

- Versions **0.x** : tout est permis (pas encore stable).
- Après 1.0 : le numéro est un **contrat** avec les utilisateurs.
- Le point n'est **pas** un séparateur décimal : après 1.9 vient 1.10.

<https://semver.org>.

Versionnement calendaire (*calver*)

Alternative, basée sur la **date** :

Projet	Format	Exemple
Ubuntu	YY.MM	24.04
pip	YY.MINOR	24.0
Chrome	MAJOR incrémental rapide	125

Pertinent quand la compatibilité arrière est moins cruciale que la fraîcheur (distributions, navigateurs).

<https://calver.org>.

Release notes et changelog

Standard « Keep a Changelog » (<https://keepachangelog.com>) :

Catégorie	Description
Added	Nouvelles fonctionnalités
Changed	Modifications (impact compatibilité possible)
Deprecated	Fonctionnalités bientôt supprimées
Removed	Fonctionnalités supprimées
Fixed	Corrections de bugs
Security	Corrections de vulnérabilités

GitHub Releases

Un `git tag` + une page GitHub dédiée avec notes de version auto-générées (à partir des PRs), fichiers attachés (binaires, docs), versions préliminaires (beta, RC), notifications aux *watchers*. Bonnes pratiques : compléter avec un `CHANGELOG.md`, créditer les contributeurs, catégoriser via les labels des PRs, automatiser via GitHub Actions.

Branches et cycles de release

Deux approches pour le rythme :

Approche	Description	Exemple
<i>Feature-based</i>	Release quand les features sont prêtes	Projets jeunes
<i>Calendar-based</i>	Cycle fixe (6 semaines, 6 mois...)	Ubuntu, Chrome, Rust

Backporting : appliquer un correctif de `main` sur une branche stable (1.0.x, 1.1.x...).

Ce qu'il faut retenir

1. **PRs** : petites, bien décrites, testées.
2. **Commits** : messages clairs, format Conventional Commits.
3. **Communication** : asynchrone, polie, complète.
4. **Review** : contribuer à la review, pas seulement la recevoir.
5. **Lancer un projet** : ouvrir tôt, documenter progressivement, choisir des outils standards.
6. **Releases** : semver, changelog, cycles prévisibles.

Checklist avant de soumettre une PR

- ☐ La PR ne contient qu'un changement logique.
- ☐ Les tests passent localement.
- ☐ Le code respecte les *guidelines* du projet.
- ☐ Le message de commit est clair.
- ☐ La description explique le *pourquoi*.
- ☐ L'issue liée est référencée.
- ☐ J'ai relu mon propre diff.

Pour aller plus loin

- Karl Fogel, *Producing Open Source Software* : <https://producingoss.com>.
- Conventional Commits : <https://www.conventionalcommits.org>.
- Semver : <https://semver.org>.
- Keep a Changelog : <https://keepachangelog.com>.

Session 8 — Gouvernance des projets open source

Objectifs de la séance

- Identifier les modèles de gouvernance des projets majeurs.
- Expliquer le **droit au fork** et ses implications.
- Analyser les mécanismes de prise de décision.
- Comprendre le parcours pour devenir mainteneur.
- Comprendre le rôle des fondations.

Deux actualités en guise d'introduction

Avant d'attaquer la théorie, deux affaires récentes (2026) illustrent les tensions de la gouvernance open source.

Euro Office vs OnlyOffice (mars 2026)

Une coalition de 8 éditeurs européens (Ionos, Nextcloud, XWiki, OpenProject, Abilian...) lance **Euro Office**, fork de la suite **OnlyOffice** sous **AGPL v3**. OnlyOffice crie à la violation de licence et invoque des conditions supplémentaires (Section 7 AGPL) exigeant de conserver logo et marque.

Ce que la Section 7 AGPL autorise réellement :

- **7(b)** — exiger l'**attribution** (créditer l'auteur) : pas forcer l'affichage d'une marque commerciale.
- **7(e)** — **refuser** les droits sur les marques : Euro Office n'utilise pas la marque OnlyOffice, elle a été retirée, ce qui est **conforme**.
- Toute condition supplémentaire non prévue par les clauses 7(a)-(f) est considérée comme une « *further restriction* » que le bénéficiaire de la licence a le **droit de supprimer** (Section 7, dernier paragraphe, confirmé par la Section 10).

Conclusion : (1) droit d'auteur ≠ droit des marques ; (2) on ne peut pas prétendre faire du logiciel libre et refuser un fork. Le fork est **légitime** juridiquement.

TDF expulse les développeurs de Collabora (avril 2026)

The Document Foundation (fondation derrière LibreOffice) expulse **30+ membres** affiliés à **Collabora**, dont **7 des 10 principaux committers** et plusieurs fondateurs du projet.

Les griefs de Collabora (Michael Meeks) : board rempli de non-techniciens, développeurs exclus de la gouvernance, application sélective des politiques de marques, manipulations d'élections. Réponse de Collabora : infrastructure de review indépendante (Gerrit), divergence accrue entre Collabora Office et LibreOffice — signal clair vers un **fork de facto**.

Ces deux cas illustrent les **tensions structurelles** (éditeur commercial vs fondation ; droit au fork vs contrôle de marque) que cette session va éclairer.

Partie 1 — Le droit au fork

Une garantie fondamentale

« *The indispensable ingredient that binds developers together on a free software project is the code's forkability.* » — Karl Fogel, *Producing Open Source Software*.

Définition : droit de copier un projet et de le développer indépendamment. Ce qui le rend possible : licence libre + code source disponible + aucune autorisation nécessaire.

Pourquoi le fork est fondamental

Pour la liberté des utilisateurs : c'est une option de sortie, une garantie d'indépendance vis-à-vis du mainteneur, une protection contre les dérives, et un gage de pérennité du projet.

Pour la gouvernance du projet : le fork joue le rôle d'épée de Damoclès — il force les mainteneurs à écouter la communauté, limite le pouvoir des « dictateurs » et constitue un mécanisme de dernier recours.

Le paradoxe du fork

« *The possibility of forks is usually a much greater force than actual forks.* »

Les vrais forks sont rares. Coûts : division de la communauté, duplication des efforts, confusion pour les utilisateurs, perte de momentum. Effet dissuasif : la **menace** suffit souvent à faire avancer les compromis.

Forks célèbres

Fork	Projet original	Raison	Résultat
LibreOffice	OpenOffice	Rachat par Oracle	Succès
MariaDB	MySQL	Rachat par Oracle	Succès
io.js	Node.js	Gouvernance	Réunification
Jenkins	Hudson	Conflit Oracle	Succès
OpenTofu	Terraform	Changement de licence (BSL)	En cours
Valkey	Redis	Changement de licence	En cours
OpenSearch	Elasticsearch	Changement de licence (soutenu par AWS)	Actif

Partie 2 — Modèles de gouvernance

Le modèle BDFL (*Benevolent Dictator For Life*)

Une personne a le dernier mot. Autorité basée sur le mérite, délégation possible. « Dictature » tempérée par le fork.

Exemples : **Linux** (Linus Torvalds), **Python** (Guido van Rossum jusqu'en 2018), **Ubuntu** (Mark Shuttleworth).

« *Only when it is clear that no consensus can be reached, and that most of the group wants someone to make a decision so that development can move on, does she put her foot down.* »

Le bon BDFL décide **rarement** par autorité, écoute la communauté, utilise son pouvoir en dernier recours, peut se retirer (comme Guido van Rossum en 2018).

Le modèle Apache (méritocratie)

Structure en cercles concentriques :

Utilisateurs → Contributeurs → Committers (écriture)
→ PMC (Project Management Committee) → Chair

Rôle	Droits	Comment l'obtenir
Utilisateur	Utiliser, reporter des bugs	Automatique
Contributeur	Proposer des patches	Automatique
Committer	Commit direct	Invitation par le PMC
PMC Member	Vote, décisions	Invitation par le PMC
PMC Chair	Représentation officielle	Élection

Principe central : les droits s'obtiennent par le **mérite**, mesuré par la qualité et la régularité des contributions.

Le modèle comité élu

Exemples modernes : **Rust**, **Python** (post-Guido, PEP 13), **Node.js**.

Caractéristiques : *Steering Committee* élu, mandats limités, processus formalisé (RFC), transparence.

Avantages : légitimité démocratique, renouvellement, évite la concentration, processus clairs.

Comparatif

Aspect	BDFL	Méritocratie Apache	Comité élu
Décisions	Rapides	Consensus	Formalisées
Succession	Difficile	Naturelle	Organisée
Scalabilité	Moyenne	Bonne	Bonne
Transparence	Variable	Moyenne	Haute
Exemples	Linux	Apache	Rust, Python

Partie 3 — Prise de décision

Le consensus paresseux (*lazy consensus*)

« *As long as nobody explicitly opposes a proposal, it is recognized as having the support of the community.* »

Fonctionnement : (1) une proposition est faite, (2) période d'attente, (3) si personne ne s'oppose → acceptée, (4) si objection → discussion. Efficace, évite la bureaucratie.

Quand le consensus échoue

Mécanismes de résolution, dans l'ordre :

1. **Discussion prolongée** — chercher un compromis.
2. **Vote** — majorité qualifiée (souvent 2/3 ou 3/4).
3. **Décision BDFL / PMC** — l'autorité tranche.
4. **Report** — attendre plus d'information.
5. **Fork** — dernier recours.

Les RFC / PEP / CEP

Processus formalisé pour les **changements majeurs** :

Projet	Nom	Description
Python	PEP	Python Enhancement Proposal
Rust	RFC	Request for Comments
Kubernetes	KEP	Kubernetes Enhancement Proposal
IETF	RFC	Internet standards

Contenu type : motivation, design, alternatives, impact.

Comparaison avec les ADR (*Architecture Decision Records*) utilisés en entreprise : même logique de documenter les décisions et leurs justifications, mais les PEP/RFC sont **publics** et soumis à la communauté.

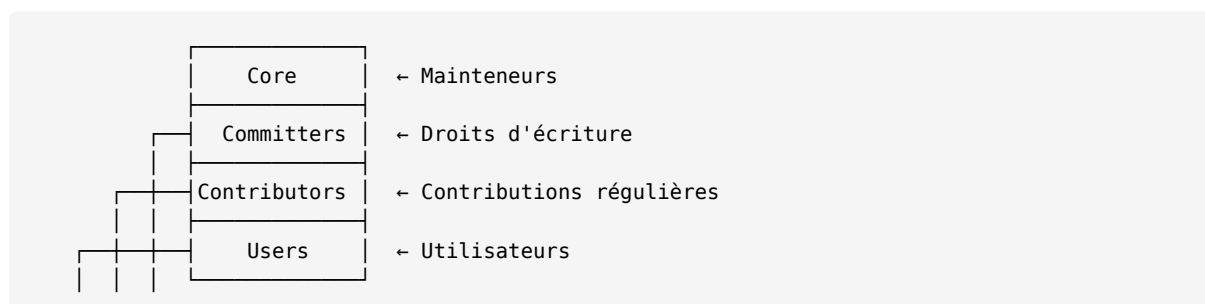
Exemple : processus PEP

1. Idée → discussion informelle
2. Draft PEP → rédaction formelle
3. Review → commentaires de la communauté
4. Décision → Accepté / Rejeté / Différé
5. Implémentation → si accepté
6. Final → documentation officielle

Types de PEP : *Standards Track* (changements techniques, ex. PEP 484 type hints), *Informational* (recommandations, ex. PEP 8 style guide), *Process* (changements de processus, ex. PEP 13 gouvernance post-Guido).

Partie 4 — Devenir mainteneur

Le modèle *onion*



Progression : de l'extérieur vers l'intérieur, **par le mérite**.

Comment devenir *committer* / mainteneur

Ce qui compte : contributions régulières et de qualité, revue de code des autres, aide aux nouveaux, participation aux discussions, fiabilité et constance dans le temps.

Le processus : cooptation, le plus souvent — proposition par un mainteneur existant, vote ou consensus du groupe, parfois une période d'essai.

Responsabilités

Techniques : revoir les PRs, merger le code, maintenir la qualité, gérer les releases, corriger les bugs critiques.

Communautaires : accueillir les nouveaux, modérer les discussions, communiquer les décisions, représenter le projet à l'extérieur.

Le *burnout* des mainteneurs

Problème croissant. Causes : charge bénévole, pression des utilisateurs (exigences sans retour), critiques non constructives, solitude. Solutions : déléguer, **dire non**, prendre des pauses, chercher des co-mainteneurs. Le *burnout* est au cœur de plusieurs incidents supply chain (cf. XZ Utils, session 10).

Partie 5 — Les fondations

Pourquoi des fondations ?

Besoins des projets : une entité légale, une structure pour détenir marques et noms de domaines, un cadre pour recevoir des dons (avec défiscalisation éventuelle), une protection juridique contre les poursuites.

Ce que les fondations apportent : neutralité (elles ne sont pas une entreprise à objectif commercial), pérennité au-delà des fondateurs, infrastructure partagée (CI, site, événements), gouvernance formelle — parfois au prix d'une certaine lourdeur.

Fondations majeures

Fondation	Style	Projets notables
Apache Software Foundation	Communautaire (<i>Apache Way</i>)	Kafka, Spark, Hadoop, Tomcat
Linux Foundation	<i>Umbrella</i> corporate, budget 500 M\$	Linux, Kubernetes, Node
Mozilla Foundation	<i>Mission-driven</i>	Firefox
Eclipse Foundation	Corporate	Eclipse IDE, Jakarta EE
CNCF (Linux Fdn)	Corporate cloud	Kubernetes, Prometheus, Helm
Python Software Foundation	Communautaire	Python, PyPI
FSF	Idéologique	GNU, GCC

Apache vs Linux Foundation

Apache Software Foundation : devise « *community over code* » ; tous les projets doivent suivre l'*Apache Way* (gouvernance uniforme) ; quasi exclusivement bénévole, staff salarié très réduit (≤ 10 personnes, essentiellement pour l'infra et l'administratif).

Linux Foundation : modèle *umbrella* — les projets hébergés conservent leur propre gouvernance ; financement massif par les grandes entreprises ; budget important (plusieurs centaines de millions de dollars) ; staff conséquent.

Deux philosophies opposées : uniformiser la gouvernance (ASF) vs héberger des projets divers sans imposer un modèle unique (LF).

Associations françaises et européennes

Rappel : **CNLL** (filière professionnelle France), **APELL** (européenne), **APRIL** (grand public), **Framasoft** (logiciels libres + services éthiques grand public), **ADULLACT** (collectivités territoriales).

Ce qu'il faut retenir

1. **Fork** : garantie fondamentale, rarement exercée mais toujours présente.
2. **Modèles** : BDFL, méritocratie Apache, comités élus.
3. **Décisions** : consensus paresseux, votes, RFC/PEP.
4. **Progression** : modèle *onion*, cooptation par le mérite.
5. **Fondations** : structures légales neutres pour les projets — mais toutes ne se valent pas (*Apache Way* vs *umbrella*).
6. **Tensions récurrentes** : éditeur commercial vs fondation (TDF/Collabora), droit au fork vs marque (Euro Office/OnlyOffice) — à suivre en session 9.

Pour aller plus loin

- Karl Fogel, *Producing Open Source Software* : <https://producingoss.com>.
- PEP 13 (gouvernance Python post-Guido) : <https://peps.python.org/pep-0013/>.
- *Apache Way* : <https://www.apache.org/theapacheway/>.

Session 9 — Modèles économiques et open source en entreprise

Objectifs de la séance

- Identifier les principaux modèles économiques de l'open source.
- Analyser les tensions entre communauté et *business*.
- Comprendre le rôle d'un **OSPO** et de l'**InnerSource**.
- Évaluer les obligations de conformité d'une entreprise.

Partie 1 — Le paradoxe de l'open source

Comment gagner de l'argent avec du « gratuit » ?

■ « *If the software is free, how do you make money?* »

Réponse courte : le logiciel est libre, pas les services associés.

Réponse longue : plusieurs modèles économiques viables existent, chacun avec ses compromis.

Ce que « gratuit » signifie vraiment

Ce qui est gratuit : le code source ; le droit d'utiliser, de modifier, de redistribuer.

Ce qui ne l'est pas forcément : le support, l'expertise, l'intégration dans un SI, les garanties, les fonctionnalités premium d'un éditeur.

L'économie de l'abondance

Dans le monde physique, rareté → valeur. Dans le monde logiciel, **abondance** : la copie est gratuite, la distribution quasi gratuite. Où est la valeur ?

- **Temps et expertise** (toujours rares).
- **Confiance et réputation**.
- **Personnalisation et intégration**.
- **Support et garanties**.

Le coût marginal du logiciel est nul

- **Coût de création** : élevé (développement, tests, documentation).
- **Coût de reproduction** : **zéro** (copier un fichier).
- **Coût de distribution** : quasi nul (Internet).

Conséquences fondamentales :

- Le logiciel est un bien **non-rival** : mon usage ne diminue pas le vôtre.
- Vendre des copies n'a de sens que grâce au **droit d'auteur** (exclusivité artificielle).
- L'open source **supprime** cette exclusivité : il faut vendre **autre chose** que la copie.
- D'où les modèles fondés sur services, support, confiance, expertise.

■ C'est cette propriété économique qui rend l'open source possible — et qui oblige à repenser les modèles de revenus.

Partie 2 — Les modèles économiques

Vue d'ensemble

Modèle	Description	Exemples
Dual licensing	GPL + licence commerciale	MySQL, Qt
Open core	Cœur libre + plugins payants	GitLab, Rudder, Centreon
Distributeur	Intégration + support	Red Hat, SUSE
Services	Consulting, formation, intégration	Smile, Oslandia
SaaS	Version hébergée	GitLab.com, Clever Cloud
Delayed OSS	Code ouvert après délai (BSL...)	MariaDB, HashiCorp
Donations / sponsors	Financement communautaire	curl, Vue.js

Dual licensing

Même code, **deux licences** :

- **Licence copyleft** (GPL) — gratuit, copyleft s'applique, pour les projets libres.
- **Licence commerciale** — payante, pas de copyleft, pour l'usage propriétaire.

Exemples : MySQL, Qt. Nécessite le **contrôle du copyright** (CLA ou cession). Perçu parfois comme « exploitation » car le projet reçoit des contributions gratuites mais seul l'éditeur peut les vendre en propriétaire.

Open core

Cœur libre + extensions propriétaires :



Exemples : GitLab (CE vs EE), Grafana, Rudder, Centreon.

Le dilemme : trop de features dans le core → pas de revenus ; trop peu → communauté frustrée.

Distributeur / support : le modèle Red Hat

Vendre du support et de l'intégration autour de logiciels 100 % libres.

Ce qui est gratuit : le code source (Fedora, CentOS Stream) ; les binaires des *rebuilt*s (Rocky Linux, AlmaLinux).

Ce qui est payant : RHEL (binaires officiels de Red Hat), support 24/7, certifications, garanties, responsabilité contractuelle.

Rachat de Red Hat par IBM (2019) : **34 Md \$**.

Services professionnels — transversal à tous les modèles

Quel que soit le modèle principal (open core, dual, SaaS...), les **services à haute valeur ajoutée** sont une source majeure de revenus pour les éditeurs.

Types de services : consulting et architecture, développement sur mesure, formation et certification, intégration et migration, support et SLA.

Pourquoi c'est important : revenus récurrents et prévisibles, fidélisation des clients, remontée de besoins du terrain, et surtout — ça marche même sans licence propriétaire.

Réputation et confiance — « Red Hat, c'est comme Evian »

« Red Hat c'est comme Evian : l'eau est gratuite. Pourtant Evian vend des milliards de bouteilles. Pourquoi ? Parce que vous faites confiance à la marque. » — Bob Young, ancien CEO de Red Hat (1998).

Ce que vend vraiment un éditeur open source :

- **Garantie** : ça marche, et si ça casse, quelqu'un répond.
- **Certification** : matériel, cloud, conformité (FIPS, Common Criteria...).
- **Réputation** : « personne ne s'est fait virer pour avoir choisi Red Hat ».
- **Responsabilité juridique** : indemnisation en cas de violation de licence.

La certification crée un **écosystème** (partenaires, compétences, emplois) qui renforce la position de l'éditeur.

SaaS et le Cloud Problem

SaaS : héberger le logiciel et vendre l'accès. Revenus récurrents, contrôle de l'expérience, scalabilité. Ex : GitLab.com, MongoDB Atlas, Clever Cloud.

Le Cloud Problem : la licence open source autorise **n'importe qui** à proposer le logiciel en service — y compris AWS, GCP, Azure.

Scénario type :

1. Un éditeur crée une BDD open source (Elasticsearch).
2. Il finance le développement, anime la communauté.
3. AWS lance *Amazon OpenSearch Service* — le même logiciel, hébergé par AWS.
4. AWS a plus de clients, plus de data centers, plus de force commerciale.
5. L'éditeur original perd des clients SaaS.

C'est **légal**, mais c'est l'éditeur qui a tout financé. « *We built it, they sell it.* » — refrain partagé par MongoDB, Elastic, Redis...

Les réponses au Cloud Problem

Réponse 1 — Changer de licence pour exclure le cloud public : SSPL (MongoDB) interdit d'offrir le logiciel comme service ; BSL (MariaDB) devient open source après un délai ; Elastic License impose des restrictions cloud. Controverse majeure : ces licences **ne sont pas open source** selon l'OSI.

Réponse 2 — Différenciation sans changement de licence : proposer des features exclusives à sa propre version SaaS (GitLab), offrir une UX supérieure, miser sur l'intégration, le support et la certification, capitaliser sur la marque et la confiance.

Licences « chrono-dégradables » (*Delayed Open Source*)

Code propriétaire ou restreint pendant un temps, puis open source :

Modèle	Mécanisme	Exemple
Ghostscript (1990s)	Version N-1 libérée quand la N sort	Précurseur historique
BSL	Conversion automatique après un délai fixe (typiquement 3-4 ans)	MariaDB, HashiCorp
Sponsorware	Features libérées quand un seuil de sponsoring est atteint	Material for MkDocs

Material for MkDocs (squidfunk) : les features *Insiders* sont réservées aux sponsors GitHub, libérées sous MIT quand un niveau de financement mensuel est atteint.

Limites : dépendance aux intermédiaires (GitHub Sponsors, Open Collective) qui prélèvent une commission et contrôlent la relation avec les sponsors.

Donations et sponsors

Plateformes de financement communautaire : GitHub Sponsors, Open Collective, Tidelift (orienté entreprise).

Financement public : NLnet (Europe), Sovereign Tech Fund (Allemagne), NGI — Next Generation Internet (UE).

Exemples de réussites : **Vue.js** (Evan You), Babel, Godot Engine.

Limite : rarement suffisant pour vivre à plein temps, sauf pour quelques « superstars ».

L'écosystème open source français

Entreprise	Modèle	Activité
Nexedi	Services + SaaS	ERP5, SlapOS
XWiki	Open core + SaaS	Wiki d'entreprise, CryptPad
Rudder	Open core	Automatisation IT / conformité
Centreon	Open core	Supervision IT
Smile	Services / intégration	Plus grand intégrateur européen
Clever Cloud	SaaS (PaaS)	Hébergement cloud
BlueMind	Open core	Messagerie collaborative
Enalean	Open core + services	Tuleap (ALM)

Structuration : **CNLL** (union des entreprises du libre et du numérique ouvert), **APELL** au niveau européen.

Partie 3 — Cas d'études et tensions

Cas 1 — HashiCorp change de licence (2023)

Août 2023 : Terraform passe de **MPL** → **BSL**. Avril 2024 : **IBM rachète HashiCorp** pour **6,4 Md \$**. La communauté réagit en créant **OpenTofu** (Linux Foundation). Le rachat peu après renforce le soupçon de « *bait and switch* » pré-acquisition.

Cas 2 — Redis et Valkey (2024)

Mars 2024 : Redis (la société, ex-Redis Labs) fait passer le **cœur de Redis** (à partir de la version 7.4) de BSD 3-clauses à un double régime **RSALv2 / SSPLv1** — donc hors open source au sens de l'OSI. Fork **Valkey** aussitôt lancé sous l'égide de la Linux Foundation, soutenu par AWS, Google, Oracle.

Cas 3 — WordPress vs WP Engine (2024)

Matt Mullenweg (BDFL WordPress, CEO Automattic) reproche à WP Engine — hébergeur WordPress majeur — de ne pas contribuer suffisamment.

Arguments de Mullenweg : WP Engine profite massivement de l'écosystème WordPress ; aucun contributeur à la *core team* ; pas de contribution financière à la fondation.

Arguments de WP Engine : ils contribuent via des plugins et l'animation de l'écosystème ; les moyens de pression employés par Automattic sont disproportionnés ; il y a confusion entre les intérêts de la fondation et ceux d'Automattic.

Question soulevée : quand une seule personne contrôle le projet **et** l'entreprise commerciale, qui arbitre ?

La tension communauté / business

Intérêts de la communauté : tout devrait être libre, gouvernance ouverte, pas de « faux » open source (source-available déguisé).

Intérêts business : revenus durables, protection contre les « parasites » (cloud providers notamment), contrôle stratégique de la roadmap.

Équilibre difficile à trouver et à maintenir.

Rappel – comment fonctionne le Venture Capital

Le VC (capital-risque) finance des startups en échange de parts du capital.

1. La startup a une **valorisation** (ex. 10 M€ *pre-money*).
2. Le VC investit (ex. 5 M€) → valorisation *post-money* = 15 M€, VC détient 33 %.
3. Le VC a des **actions de préférence** (liquidation prioritaire, anti-dilution).
4. L'objectif du VC : un **exit** (revente ou IPO) avec un multiple d'au moins ×10 en 5-7 ans.

Conséquence pour l'open source : le VC n'investit pas par philanthropie. Il exige une croissance agressive et un exit lucratif. Si le modèle open source ne génère pas assez de revenus → pression pour changer la licence ou se faire racheter.

Le *bait and switch*

Pattern récurrent chez les éditeurs VC-funded :

1. Lancer un projet open source → adoption rapide, communauté gratuite.
2. Lever des fonds (VC) sur la base de cette adoption.
3. Construire un produit commercial autour du projet.
4. **Changer la licence** une fois établi (SSPL, BSL, Elastic License...).
5. Monétiser la communauté captive — ou se faire racheter (exit pour le VC).

Exemples : MongoDB (IPO), Elastic (IPO), HashiCorp (racheté IBM), Redis, CockroachDB, Sentry...

Signes d'alerte

Projet potentiellement à risque	Projet plus sûr
<i>Single-vendor</i>	Multi-vendor
CLA exigeant (<i>copyright assignment</i>)	Fondation neutre
Financement VC important	DCO simple
Pas de gouvernance communautaire	Gouvernance transparente

VC-funded vs bootstrapped

VC-funded → pression de sortie	Bootstrapped → stabilité
Croissance rapide exigée	Croissance organique
Exit (IPO ou rachat) attendu	Pas de pression actionnariale
Changement de licence fréquent	Licence stable dans le temps
Ex : MongoDB, HashiCorp, Elastic	Ex : XWiki, Nexedi, Enalean

Les **projets communautaires** (PostgreSQL, Linux) évitent la pression capitaliste mais ont d’autres risques : désengagement des contributeurs faute de modèle économique (Apache Attic, Apache OpenOffice après le retrait d’IBM) ou conflits fondation vs éditeur principal (TDF vs Collabora, cf. session 8).

Open source en entreprise

Face à ces enjeux — modèles économiques, tensions communautaires, risques de conformité — les entreprises doivent **s’organiser**. Trois réponses complémentaires :

- 1. **Conformité** — respecter les obligations légales des licences.
- 2. **OSPO** — gouverner l’usage et la contribution open source.
- 3. **InnerSource** — appliquer les pratiques open source en interne.

Partie 4 — Conformité

Obligations selon les licences

■ Rappel des sessions 4-5, appliqué au contexte entreprise.

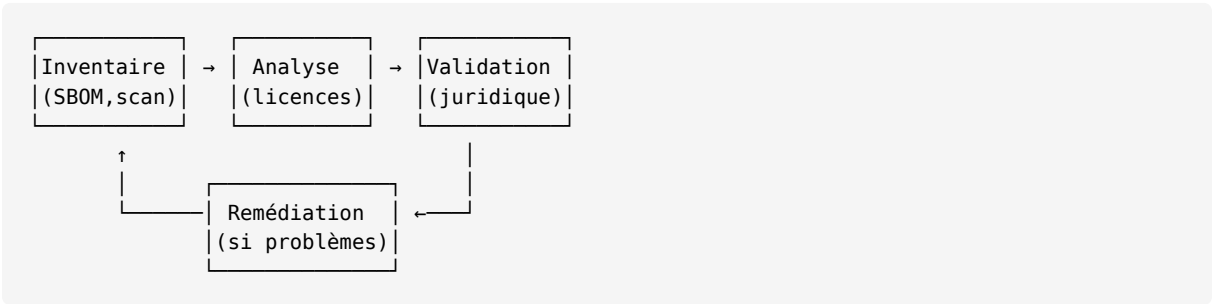
Licences permissives (MIT, BSD, Apache...) : conserver les notices de copyright, inclure le texte de la licence, parfois afficher un disclaimer de garantie.

Licences copyleft (GPL, LGPL, AGPL...) : mêmes obligations que le permissif, plus fournir le code source des modifications et conserver la même licence dans toute redistribution.

Risques de non-conformité

Entreprise	Violation	Conséquence
Cisco (2008)	GPL dans Linksys	Procès FSF, <i>settlement</i>
Free (2011)	GPL Freebox	Condamné (France), code publié
VMware (2015)	Linux kernel	Procès, abandonné
Hancom (2016)	GPL Ghostscript	Condamné (Corée)
SFC vs Vizio (2021)	GPL	<i>Settlement</i> 2025
Orange (2011→2024)	GPL v2 (Lasso)	Condamné à 800 k€ (France)

Processus et outils



Outil	Type	Fonctionnalités
REUSE (FSFE)	Open source	Vérification métadonnées licence/copyright par fichier
FOSSology	Open source	Scan, identification, reporting

Outil	Type	Fonctionnalités
ScanCode	Open source	Détection de licences
Black Duck	Commercial	Enterprise, compliance
FOSSA	Commercial	Intégration CI/CD

Partie 5 – L’OSPO (Open Source Program Office)

Qu’est-ce qu’un OSPO ?

Structure dédiée à la **gestion de l’open source** en entreprise.

Missions principales : définir la politique d’utilisation, s’assurer de la conformité des licences, coordonner la contribution *upstream*, animer les relations avec les communautés externes.

Rattachement hiérarchique typique : CTO ou direction technique, direction juridique, ou structure transverse indépendante. L’OSPO travaille en transverse, au service de toutes les équipes.

Pourquoi créer un OSPO ?

Enjeux pour l’entreprise :

1. **Juridiques** — conformité, gestion des risques.
2. **Sécurité** — *supply chain*, vulnérabilités (cf. session 10).
3. **Stratégiques** — influence sur les standards, recrutement, innovation.
4. **Économiques** — optimisation des coûts, éviter le *vendor lock-in*.

Les entreprises utilisent en moyenne **500+** composants open source.

Responsabilités

Côté consommation d’open source : inventaire des dépendances, validation des licences, gestion des vulnérabilités, formation des développeurs.

Côté contribution : définir une politique de contribution interne, mettre en place un processus de validation, entretenir les relations avec les projets amont, financer (sponsoring, donations).

Exemples

Entreprise	OSPO depuis	Focus
Google	2004	Innovation, Android, Kubernetes
Microsoft	2014	Transformation, GitHub
Red Hat	Origine	Cœur de métier
SAP	2017	Conformité, contribution
Spotify	2018	InnerSource, outils
SNCF	2020	Souveraineté

Communauté des OSPO : **TODO Group** (Linux Foundation) — <https://todogroup.org/>.

Partie 6 – InnerSource

Qu’est-ce que l’InnerSource ?

Appliquer les pratiques open source à l’intérieur de l’entreprise.

Open Source	InnerSource
Code public	Code interne partagé
Contributeurs externes	Contributeurs = employés
Communauté mondiale	Communauté entreprise
Gouvernance ouverte	Gouvernance adaptée

Problèmes résolus

Situation typique :

- Équipe A développe un composant.
- Équipe B a besoin d'une modification.
- Équipe A est occupée → ticket dans le backlog.
- Équipe B attend... ou duplique le code.

Avec InnerSource : équipe B propose une PR, équipe A review et merge. Tout le monde gagne du temps.

Rôles et mise en place

Rôle	Responsabilités
Guest	Contributeur externe à l'équipe
Trusted Committer	Mainteneur, <i>review</i> , mentorat
Product Owner	Vision, priorités, <i>roadmap</i>
Contributor	Développeur qui soumet des PRs

Étapes recommandées :

1. **Pilote** — commencer avec 2-3 projets volontaires.
2. **Outillage** — forge interne (GitLab, GitHub Enterprise).
3. **Culture** — valoriser les contributions *cross-équipes*.
4. **Métriques** — mesurer l'adoption et les bénéfices.

Ressources : **InnerSource Commons** — <https://innersourcecommons.org/>.

Ce qu'il faut retenir

1. **Coût marginal nul** → l'open source oblige à vendre autre chose que la copie.
2. **Plusieurs modèles** viables : services, open core, dual licensing, SaaS, delayed OSS, sponsors.
3. **Réputation et certification** : la confiance est le vrai produit (Bob Young, Red Hat).
4. **Cloud Problem** et **VC-funded** expliquent la plupart des *relicensing* récents.
5. **Conformité** : obligations réelles, risques juridiques, outils disponibles.
6. **OSPO** : structure clé pour gouverner l'open source en entreprise.
7. **InnerSource** : appliquer les pratiques open source en interne.

Pour aller plus loin

- *Open Source Business Models* (OSI) — <https://opensource.org>
- CNLL — <https://cnll.fr>
- TODO Group (OSPO) — <https://todogroup.org>
- InnerSource Commons — <https://innersourcecommons.org>

Session 10 — Supply chain et sécurité

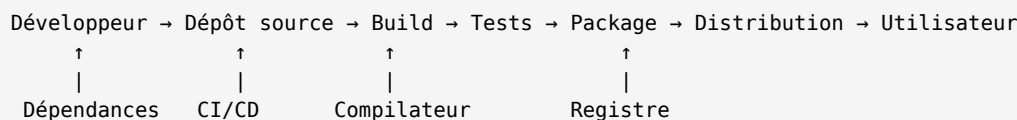
Objectifs de la séance

- Comprendre ce qu'est une *supply chain* logicielle.
- Identifier les risques de sécurité spécifiques à l'open source.
- Expliquer ce qu'est un **SBOM** et son importance.
- Analyser les incidents majeurs (**Log4Shell**, **XZ Utils**).
- Positionner les obligations du **CRA** (fabricant, distributeur, *steward*).
- Appréhender l'impact des **LLM** sur la sécurité open source.

Partie 1 — La supply chain logicielle

Définition

Ensemble des composants, outils, processus et personnes impliqués dans le développement et la distribution d'un logiciel.



La réalité des dépendances

Dépendances directes : les bibliothèques que vous importez explicitement, les frameworks utilisés, les outils de build.

Dépendances transitives : les dépendances de vos dépendances. Souvent 10× plus nombreuses, moins visibles et plus risquées.

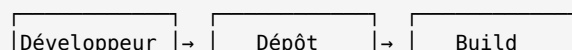
Exemple : une application React simple peut avoir 500+ dépendances npm.

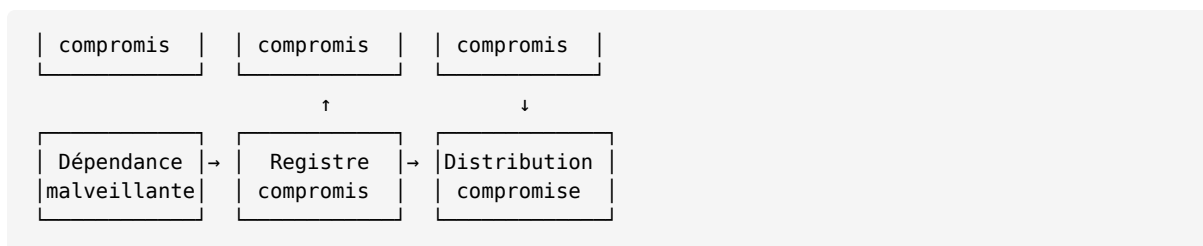
Statistiques

Écosystème	Packages	Dépendances moyennes
npm	2+ M	5-10 directes, 50-100 transitives
PyPI	500 k+	3-5 directes, 20-50 transitives
Maven	500 k+	10-20 directes, 100+ transitives
Cargo	120 k+	Variable

Plus de 90 % du code d'une application moderne provient de dépendances open source.

Points d'attaque





Partie 2 — Incidents majeurs

Log4Shell (décembre 2021)

CVE-2021-44228, score **CVSS 10.0** (critique).

La vulnérabilité : Log4j, bibliothèque de logging Java utilisée par des millions d'applications. Une injection JNDI permet l'exécution de code à distance.

L'impact : exploitation triviale (une simple chaîne dans n'importe quel champ loggé) ; découverte le 24 novembre 2021 ; divulgation publique le 9 décembre 2021 ; chaos mondial pendant des semaines.

Leçons :

1. **Invisibilité des dépendances** — beaucoup ne savaient pas utiliser Log4j (dépendance transitive).
2. **Mainteneur unique** — projet critique, quelques bénévoles.
3. **Temps de réponse** — difficulté à identifier et patcher tous les systèmes.
4. **Dépendances transitives** — Log4j était souvent indirect.

XZ Utils (mars 2024)

La porte dérobée la plus sophistiquée jamais découverte dans l'open source.

Chronologie : 2021, « Jia Tan » commence à contribuer ; 2022-2023, il gagne la confiance ; 2024, il devient co-mainteneur ; mars 2024, la backdoor est découverte.

Sophistication de l'attaque : ingénierie sociale étalée sur 3 ans visant le mainteneur surchargé ; code obfusqué caché dans les fichiers de tests ; cible SSH sur les systèmes Debian/Fedora ; découverte par hasard — un ingénieur Microsoft (Andres Freund) repère une latence anormale de 500 ms.

Leçons :

1. **Social engineering** — l'attaquant a manipulé le mainteneur surchargé (burnout).
2. **Single maintainer** — un seul mainteneur pour une lib critique.
3. **Confiance aveugle** — les distributions incluent automatiquement.
4. **Détection difficile** — découvert par un ingénieur Microsoft (Andres Freund) par chance.

■ « *We're all just one mass psychosis away from disaster.* »

Autres incidents notables

Incident	Année	Type	Impact
Heartbleed (OpenSSL)	2014	Buffer over-read	Fuite de mémoire serveur, clés TLS
leftpad	2016	Suppression d'un paquet npm de 11 lignes	Milliers de builds cassés
event-stream	2018	Mainteneur malveillant	Vol de wallets Bitcoin (Co-pay)
ua-parser-js	2021	Compte npm compromis	Cryptominers

Incident	Année	Type	Impact
colors / faker	2022	Sabotage par l’auteur	Boucle infinie, protestation non-rémunération
node-ipc	2022	Protestware (guerre Ukraine)	Fichiers écrasés sur IP russes/biélorusses
Shai Hulud	2025	Ver auto-propageant npm	Exfiltration de tokens
PyPI typosquatting	continu	Faux packages	Malware divers

Partie 3 – SBOM et inventaire

Qu’est-ce qu’un SBOM ?

Software Bill of Materials — la liste des ingrédients d’un logiciel.

Ce que contient un SBOM : tous les composants utilisés, leurs versions, leurs licences, leurs dépendances, leur provenance.

Formats standards : **SPDX** (Linux Foundation, normalisé ISO/IEC 5962:2021), **CycloneDX** (projet OWASP, normalisé ECMA-424), **SWID** (ISO/IEC).

Exemple (CycloneDX)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "components": [{
    "type": "library",
    "name": "log4j-core",
    "version": "2.14.1",
    "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",
    "licenses": [{"license": {"id": "Apache-2.0"}}]
  }]
}
```

Pourquoi le SBOM est devenu incontournable

- **Executive Order 14028** (USA, mai 2021) : les fournisseurs de logiciels au gouvernement fédéral doivent fournir un SBOM.
- **Cyber Resilience Act** (UE, publié nov. 2024, applicable **11 déc. 2027**) : premier règlement européen encadrant les produits numériques, **y compris l’open source**. 36 mois d’adaptation.

SBOM = Sécurité + Conformité + Soutenabilité :

1. **Sécurité** — détection d’impact immédiate lors d’une nouvelle faille (type Log4Shell).
2. **Conformité CRA** — preuve de la maîtrise de la supply chain.
3. **Soutenabilité** — identification des dépendances critiques à soutenir.

CRA : trois rôles, trois régimes

Rôle	Qui ?	Obligations principales
Fabricant	Met un produit sur le marché (éditeur, intégrateur)	Marquage CE, SBOM, déclaration de conformité, gestion des vulnérabilités, support ≥ 5 ans

Rôle	Qui ?	Obligations principales
Distributeur	Revend le produit	Vérifier la conformité du fabricant, coopérer en cas d'incident
Open Source Steward	Personne morale qui soutient un projet OSS (fondation, éditeur contributeur)	Politique de cybersécurité documentée, coordination de la divulgation, signalement d'incidents graves

Nouveauté : le rôle d'**Open Source Steward** (« intendant » en français) est créé **spécifiquement** par le CRA — régime allégé par rapport au fabricant.

CRA : l'exception open source

Principe : seuls les produits distribués **dans un cadre commercial** sont soumis au CRA.

Critères d'activité non commerciale (hors champ du CRA) : distribution gratuite et sans but lucratif, financement par dons ou subventions, absence de services payants associés.

Stratégie de publication parallèle : l'appréciation se fait **par produit mis sur le marché**, pas par projet. Un même code peut donc donner lieu à deux régimes : la version OSS non commercialisée relève du rôle de *Steward*, la version commerciale (support, SaaS, distribution payante) relève du rôle de **Fabricant**.

Un projet **purement communautaire** (pas d'éditeur derrière) est **hors champ** du CRA. Mais dès qu'une entreprise le met sur le marché (même modifié), elle devient fabricant pour cette version.

Guide CNLL / inno³ : <https://code.inno3.eu/ouvert/guide-cra>.

CRA : ce que ça change pour les éditeurs

Transformations internes

1. Articuler politique open source et politique cyber — convergentes mais distinctes.
2. Connaître l'**intégralité** des composants intégrés (développements internes, sous-traitance, dépendances transitives).
3. Maintenir la conformité sur toute la durée de vie du produit (≥ 5 ans).

Transformations externes

- Clauses contractuelles : obtention des SBOM en amont, reversement *upstream* des correctifs.
- Passer d'une logique d'**usage** passif de l'OSS à un rôle **proactif de contribution** (cf. session 9).

■ « *I am not your supplier.* » — **Thomas Depierre**, mainteneur OSS.

Il n'y a pas de relation contractuelle entre mainteneurs et utilisateurs. Le CRA force les fabricants à l'assumer — ou à contribuer aux projets dont ils dépendent.

Maturité open source des organisations

Niveau	Caractéristiques
OSS subi	Usage opportuniste, informatique « grise », pas de politique
OSS maîtrisé	Risques encadrés, validation des usages, SBOM générés
OSS décidé	Engagement stratégique, contributions <i>upstream</i> , enjeu métier et RH

Le CRA est un accélérateur : il force le passage du niveau 1 au niveau 2, et récompense le niveau 3 (mutualisation des coûts de cybersécurité via les communautés).

Souveraineté numérique

L’open source est un levier clé de **souveraineté numérique** — la capacité à maîtriser ses infrastructures et ses données.

Risques de dépendance : deux textes américains, aux champs distincts mais convergents, fragilisent la confidentialité des données confiées à des acteurs US.

- Le **CLOUD Act** (*Clarifying Lawful Overseas Use of Data Act*, 2018) : les autorités américaines peuvent contraindre une entreprise soumise au droit US à fournir les données qu’elle détient, **où qu’elles soient stockées dans le monde** — y compris dans une filiale européenne, et sans notification de l’utilisateur.
- **FISA Section 702** (*Foreign Intelligence Surveillance Act*, §702 ajouté en 2008, reconduit en 2024) : autorise la **surveillance de masse** — sans mandat individuel — des communications électroniques de personnes non-américaines, via les grands *electronic communications service providers* américains (opérateurs cloud, messageries...). C’est la base juridique des programmes révélés par Snowden en 2013 (**PRISM**).

Ajouter à cela le *vendor lock-in* (dépendance à un éditeur unique, propriétaire ou *source-available*) et la perte de contrôle sur les mises à jour, la roadmap et les conditions d’usage.

Conséquence pour une organisation européenne : un simple hébergement chez un *hyperscaler* US, **même en datacenter européen**, expose potentiellement les données à ces deux régimes — un point régulièrement soulevé par la CNIL et l’EDPB, notamment dans les suites de l’arrêt **Schrems II** (2020) qui a invalidé le *Privacy Shield*.

L’open source comme réponse : code auditable et modifiable ; pas de dépendance à un éditeur unique ; hébergement sur une infrastructure souveraine possible ; conformité RGPD et NIS2 facilitée.

Réglementation européenne

Texte	Portée	Impact open source
RGPD (2018)	Données personnelles	Exige la maîtrise des données — favorise l’hébergement souverain
NIS2 (2024)	Sécurité des réseaux et SI	Obligations de gestion des risques supply chain
CRA (2027)	Cyber-résilience des produits	SBOM obligatoire, gestion des vulnérabilités
Sovereignty Package (annoncé mai 2026)	Paquet européen	Devrait inclure une stratégie « Open Source » renforcée

Outils de génération de SBOM

Outil	Formats	Langages
Syft (Anchore)	SPDX, CycloneDX	Multi
Trivy (Aqua)	SPDX, CycloneDX	Multi
cdxgen	CycloneDX	Multi
pip-audit	CycloneDX	Python
uv export –format cyclonedx1.5	CycloneDX	Python
npm audit	Propriétaire	JavaScript

Partie 4 — Gestion des vulnérabilités

Bases de données

Bases officielles : **CVE** (MITRE) — attribue les identifiants universels ; **NVD** (NIST) — fournit les scores CVSS et l'analyse ; **GHSA** (GitHub Security Advisories).

Bases spécialisées OSS : **OSV** (Google), Snyk Vulnerability DB, Sonatype OSS Index.

Alerte récente : en avril 2025, l'administration Trump (DOGE) a coupé les budgets de MITRE, menaçant le fonctionnement de la base CVE — illustration de la **dépendance** du monde entier à une infrastructure US. Cf. <https://www.iisf.ie/CVE-System-grinds-to-a-near-halt>.

Le score CVSS

Common Vulnerability Scoring System.

Score	Sévérité	Exemple
0.0	Aucune	—
0.1-3.9	Faible	Info disclosure mineur
4.0-6.9	Moyenne	DoS partiel
7.0-8.9	Haute	RCE avec conditions
9.0-10.0	Critique	RCE sans auth (Log4Shell)

Facteurs : vecteur d'attaque, complexité, privilèges requis, impact sur la confidentialité/intégrité/disponibilité.

Outils de scan (SCA — *Software Composition Analysis*)

- **Dependabot** (GitHub) — alertes + PRs de mise à jour automatiques.
- **Snyk**, **OWASP Dependency-Check**, **Trivy**, **Grype**.

Exemple de configuration Dependabot :

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule: { interval: "weekly" }
    open-pull-requests-limit: 10
```

Partie 5 — Bonnes pratiques

Le problème du mainteneur solitaire

« *Imaginez que toute l'infrastructure numérique mondiale dépend d'un projet maintenu par un type au hasard dans le Nebraska.* » — Paraphrase du [XKCD #2347](#) (« Dependency »).

C'est exactement ce qui s'est passé avec Log4j (2 mainteneurs bénévoles), XZ Utils (1 mainteneur en *burnout*), OpenSSL avant Heartbleed (1 développeur à temps plein pour la lib TLS la plus utilisée au monde).

Conséquence : la sécurité de la supply chain a mis en lumière un problème structurel de **financement** et de **soutenabilité** de l'open source (cf. session 9).

Pour les développeurs

Gestion des dépendances : minimiser le nombre, vérifier que chacune est activement maintenue, verrouiller les versions avec des *lock files*, auditer régulièrement les dépendances installées.

Sécurité : activer Dependabot ou Snyk, mettre à jour rapidement les versions vulnérables, ne pas ignorer les alertes, vérifier soigneusement les nouvelles dépendances avant de les adopter.

Évaluer une dépendance

1. **Maintenance** — dernière release ? Issues traitées ?
2. **Communauté** — nombre de contributeurs ? *Bus factor* ?
3. **Sécurité** — CVE passées ? Temps de réponse ?
4. **Nécessité** — vraiment utile ? Alternative possible ?
5. **Licence** — compatible avec votre projet ?

Outils : **OpenSSF Scorecard**, Snyk Advisor, Libraries.io.

OpenSSF Scorecard

Évalue **automatiquement** la sécurité d'un projet.

Critère	Description
Binary-Artifacts	Pas de binaires dans le repo
Branch-Protection	Protection des branches
Code-Review	Reviews obligatoires
Dangerous-Workflow	Pas de workflows risqués
Maintained	Activité récente
Signed-Releases	Signatures cryptographiques
Vulnerabilities	Pas de vulnérabilités connues

Référence : <https://scorecard.dev/>.

OWASP — la communauté qui se prend en main

Open Worldwide Application Security Project — fondation à but non lucratif créée en 2001, dédiée à la sécurité des applications web.

Modèle d'organisation : communauté ouverte de bénévoles ; contenu et outils publiés sous licences libres ou Creative Commons ; financement par dons, adhésions, événements ; présence mondiale via des *chapters* locaux.

Publications phares : **OWASP Top 10** (les 10 risques web les plus courants — référence mondiale), **ASVS** (standard de vérification sécurité), **Cheat Sheets** (guides pratiques), **CycloneDX** (format SBOM).

Outils : **ZAP** (scanner web), Dependency-Check, DefectDojo, entre autres.

OWASP est un **contre-modèle vertueux** : une communauté qui se structure pour produire elle-même les ressources de sécurité que l'industrie seule ne produisait pas. À distinguer d'**OpenSSF** (Linux Foundation), qui se concentre sur la sécurité de l'open source lui-même (supply chain, Scorecard, SLSA).

Pour les organisations

Côté politique : registre de dépendances approuvées, processus de validation avant adoption, SLA de mise à jour, formation continue des développeurs.

Côté outillage : SCA (*Software Composition Analysis*) intégré dans la CI/CD, génération automatique de SBOM, monitoring continu, plan de réponse aux incidents.

Builds reproductibles

Principe : à partir du même code source, le build produit un résultat **bit-à-bit identique**, quel que soit l'environnement.

Pourquoi c'est important :

- Permet de **vérifier** qu'un binaire distribué correspond bien au code source.
- Détecte les altérations malveillantes dans le processus de build (cf. XZ Utils).
- Rend le *Trusting Trust* de Ken Thompson (1984) vérifiable.

Projet de référence : <https://reproducible-builds.org/> — Debian, Arch Linux, NixOS y travaillent activement.

Le framework SLSA

Supply-chain Levels for Software Artifacts — quatre niveaux de garantie d'intégrité :

Niveau	Exigences
SLSA 1	Documentation du processus de build
SLSA 2	Build hébergé, provenance signée
SLSA 3	Build isolé, provenance non-falsifiable
SLSA 4	Build hermétique, double vérification

Les cooldowns de dépendances

Idée simple : attendre quelques jours avant d'installer une nouvelle version d'un paquet.

Pourquoi ça marche : dans la plupart des attaques récentes (Ultralytics, Nx, rspack, Shai Hulud...), la fenêtre entre publication du code malveillant et détection par la communauté est **< 1 semaine**. Un cooldown de 7 à 14 jours aurait bloqué la quasi-totalité de ces attaques.

Adoption massive en 2025-2026 :

- **pnpm, Yarn, Bun, uv** (Python), **npm** — tous ont introduit un paramètre `minimumReleaseAge` / `--exclude-newer`.
- **Cargo, Go, NuGet** : en cours.

Exemple (`uv`) : `uv add --exclude-newer=7d requests`.

À retenir : c'est gratuit, facile, et probablement la mitigation la plus efficace contre les attaques supply chain modernes.

Sources : William Woodruff — blog.yossarian.net ; Cal Paterson — calpaterson.com/deps.html.

Bonnes pratiques — l'exemple Astral

Astral (éditeur de `uv` et `ruff`, largement utilisés en Python) a publié son *playbook* sécurité. Quelques principes simples :

Protéger le code et le CI : authentification forte (2FA matérielle), droits minimaux par défaut, versions précises des outils utilisés dans le CI (pas de mise à jour automatique silencieuse).

Protéger les releases : supprimer les mots de passe et tokens *long-lived* ; signer chaque release avec une preuve vérifiable d'origine ; double approbation humaine avant toute publication.

Côté dépendances : minimiser, appliquer les cooldowns, contribuer aux projets amont dont on dépend, les financer si possible.

Source : <https://astral.sh/blog/open-source-security-at-astral>.

Partie 6 — IA et sécurité open source

Les LLM rebattent les cartes

Les modèles de langage transforment la recherche de vulnérabilités et la *supply chain* — pour le meilleur et pour le pire.

Opportunités : détection automatisée de bugs et vulnérabilités, génération de patches, audit de code à grande échelle, aide aux mainteneurs pour le triage et la revue.

Risques nouveaux : *slop vulnerability reports* — signalements de masse souvent faux, qui saturent les mainteneurs ; ingestion non consentie du code par les modèles ; *slopsquatting* — dépendances « hallucinées » par les LLM, variante du *typosquatting* (étude Lasso Security : **20 %** des paquets cités par LLM n'existent pas) ; agents autonomes exploitant à grande échelle la surface d'attaque.

Controverse cal.com (avril 2026)

Cal.com (alternative open source à Calendly) annonce une **fermeture partielle** de son code.

Motivations invoquées :

- Flot de faux rapports de vulnérabilités générés par LLM — temps mainteneur gaspillé.
- Concurrents utilisant les LLM pour cloner la base de code rapidement.
- Difficulté à monétiser un OSS totalement ouvert face à ces pressions.

Ce que ça illustre :

- Fragilité du contrat social de l'open source face à l'IA.
- Lien avec les tensions vues en session 9 (VC, exit, *bait-and-switch*) — peut-être un prétexte.
- Précédent inquiétant : d'autres projets pourraient suivre.

Source : <https://www.strix.ai/blog/cal-com-is-closing-its-code-due-to-ai-threats>.

Ce qu'il faut retenir

1. **Supply chain** : 90 %+ du code provient de dépendances open source.
2. **Incidents** : Log4Shell et XZ Utils montrent les risques — technique (RCE) et humain (social engineering).
3. **SBOM** : inventaire obligatoire (CRA, Executive Order) — **Sécurité + Conformité + Soutenabilité**.
4. **CRA** : trois rôles (**Fabricant, Distributeur, Open Source Steward**), exception OSS non commercial.
5. **Réglementation** : RGPD, NIS2 — en tension avec le CLOUD Act et le FISA américain ; l'open source comme contre-mesure et levier de souveraineté.
6. **IA** : nouveaux risques (slop reports, cal.com, slopsquatting) et nouvelles bonnes pratiques (Astral, cooldowns).
7. **Outils et pratiques** : Dependabot, Trivy, cooldowns, attestations, minimiser, contribuer *upstream*.

Le paradoxe de la sécurité open source

Avantages : code auditable, corrections rapides, communauté vigilante, transparence totale du fonctionnement.

Risques : mainteneurs non payés (et donc vulnérables au *burnout*, à l'ingénierie sociale, etc.) ; surface d'attaque visible par les attaquants ; confiance implicite par défaut envers les dépendances ; dépendances transitives difficiles à inventorier et à auditer.

■ « *Given enough eyeballs, all bugs are shallow* » — mais qui regarde ?

Pour aller plus loin

Réglementation

- Texte officiel CRA : <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>
- Guide CRA CNLL / inno³ : <https://code.inno3.eu/ouvert/guide-cra>

Standards et frameworks

- **SPDX** : <https://spdx.dev>
- **CycloneDX** : <https://cyclonedx.org>
- **OpenSSF Scorecard** : <https://scorecard.dev>
- **SLSA Framework** : <https://slsa.dev>
- **Reproducible Builds** : <https://reproducible-builds.org>

Communautés et fondations sécurité

- **OWASP** : <https://owasp.org>
- **OpenSSF** (Linux Foundation) : <https://openssf.org>

Financement de l'OSS

- **Sovereign Tech Fund** (Allemagne) : <https://www.sovereign.tech>
- **Open Source Pledge** : <https://opensourcepledge.com>